

Lab5

宋昊谦 尹浩燃 穆浩宁

练习0：填写已有实验

在已有的基础上，做一些修改：

新增 do_pgfault 函数

在 Lab 5 的初始代码框架中，内核并未提供完整的缺页异常处理函数 `do_pgfault`，且 `trap.c` 中的异常分发逻辑对于 `PAGE_FAULT` 类异常（如指令、加载、存储缺页）仅输出调试信息而未进行实质处理。这意味着，一旦发生页面缺失（Page Fault），无论是由于用户栈的自动增长、合法的按需分页请求，还是由写时复制（COW）触发的权限异常，内核都无法正确处理。这些异常只会触发默认的错误输出或导致系统 Panic，无法支持用户进程的正常运行。这导致在测试 `faultread`、`faultreadkernel` 和 `testbss` 等用例时，执行时间过长。

为此，我们实现了 `do_pgfault` 函数，并在 `trap.c` 中将其挂载到对应的异常处理分支上。

A. 异常分发 (trap.c 修改)

我们在 `exception_handler` 中针对以下三种异常类型调用了 `do_pgfault`：

- `CAUSE_FETCH_PAGE_FAULT` (取指缺页)
- `CAUSE_LOAD_PAGE_FAULT` (读数据缺页)
- `CAUSE_STORE_PAGE_FAULT` (写数据缺页)

代码逻辑如下：

```
// 以 CAUSE_STORE_PAGE_FAULT 为例
case CAUSE_FETCH_PAGE_FAULT:
    cprintf("Instruction page fault\n");
    // 调用 do_pgfault，参数：当前进程mm，错误原因，错误地址(tval)
    if (do_pgfault(current->mm, tf->cause, tf->tval) != 0) {
        print_trapframe(tf);
        if (current == NULL) {
            panic("handle_exception: page fault in kernel (current == NULL)");
        }
        do_exit(-E_KILLED);
    }
```

```

    }
    break;

case CAUSE_LOAD_PAGE_FAULT:
    cprintf("Load page fault\n");
    // 调用 do_pgfault
    if (do_pgfault(current->mm, tf->cause, tf->tval) != 0) {
        print_trapframe(tf);
        if (current == NULL) {
            panic("handle_exception: page fault in kernel (current == NULL)");
        }
        do_exit(-E_KILLED);
    }
    break;

case CAUSE_STORE_PAGE_FAULT:
    cprintf("Store/AMO page fault\n");
    // 调用 do_pgfault
    if (do_pgfault(current->mm, tf->cause, tf->tval) != 0) {
        print_trapframe(tf);
        if (current == NULL) {
            panic("handle_exception: page fault in kernel (current == NULL)");
        }
        do_exit(-E_KILLED);
    }
    break;

```

B. do_pgfault 核心逻辑 (kern/mm/vmm.c)

```

int do_pgfault(struct mm_struct *mm, uint32_t error_code, uintptr_t addr) {
    int ret = -E_INVAL;
    struct vma_struct *vma = find_vma(mm, addr);

    pgfault_num++;

    if (vma == NULL || vma->vm_start > addr) {
        return -E_INVAL;
    }

    // 权限检查: 如果尝试写一个不可写的 VMA, 直接报错
    if ((error_code & 2) && !(vma->vm_flags & VM_WRITE)) {
        return -E_INVAL;
    }
}

```

```

uint32_t perm = PTE_U;
if (vma->vm_flags & VM_WRITE) {
    perm |= (PTE_R | PTE_W);
}
if (vma->vm_flags & VM_READ) {
    perm |= PTE_R;
}
if (vma->vm_flags & VM_EXEC) {
    perm |= PTE_X;
}

addr = ROUNDDOWN(addr, PGSIZE);
ret = -E_NO_MEM;
pte_t *ptep = NULL;

// 获取 PTE，如果不存在(PT未分配)则分配
if ((ptep = get_pte(mm->pgdir, addr, 1)) == NULL) {
    return ret;
}

// Case 1: 页表项全为 0，说明尚未建立映射 (Demand Paging)
if (*ptep == 0) {
    if (pgdir_alloc_page(mm->pgdir, addr, perm) == NULL) {
        return ret;
    }
}

// Case 2: 页表项存在，可能是 COW 或者 Swap (本实验暂不考虑 swap)
else {
    if (*ptep & PTE_V) {
        // LAB5 CHALLENGE: Copy on Write 处理
        // 判断条件：这是写操作 (error_code & 2)
        //      && 且物理页目前是只读的 (!(*ptep & PTE_W))
        //      && 且 VMA 允许写入 (vma->vm_flags & VM_WRITE)
        if ((error_code & 2) && !(*ptep & PTE_W) && (vma->vm_flags & VM_WRITE)) {
            struct Page *page = pte2page(*ptep);

            // 情况 A: 页面被多个进程共享 (Reference Count > 1)
            // 需要执行“复制”：分配新页，拷贝内容，重新映射
            if (page_ref(page) > 1) {
                struct Page *npage = alloc_page();
                if (npage == NULL) return ret;
            }
        }
    }
}

```

```

// 复制原页面内容到新页面
memcpy(page2kva(npag), page2kva(page), PGSIZE);

// 建立新映射:
// 1. page_insert 会把虚拟地址 addr 映射到 npag
// 2. npag 的 ref 会增加
// 3. 原 page 的 ref 会自动减少 (因为该地址原先映射的页被覆盖了)
// 4. 注意: 这里赋予了 PTE_W 写权限
if (page_insert(mm->pgdir, npag, addr, perm) != 0) {
    return ret;
}
}
// 情况 B: 页面只有当前进程在使用 (Reference Count == 1)
// 说明其他共享的进程已经退出, 或者经过多次 COW 后只剩自己
// 只需要恢复写权限即可, 无需拷贝
else {
    // page_insert 会检测到是同一个页, 只更新权限并刷新 TLB
    if (page_insert(mm->pgdir, page, addr, perm) != 0) {
        return ret;
    }
}
} else {
    // 如果不是 COW 情况的权限错误, 则返回错误
    return ret;
}
}
}
return 0;
}

```

该函数承担了以下核心职责:

1. VMA 查找与权限检查:

- 通过 `find_vma` 确认发生缺页的虚拟地址 (`addr`) 是否位于合法的虚拟内存区域 (VMA) 内。
- **权限校验:** 如果是写操作 (`error_code & 2`), 但对应的 VMA 不允许写入 (`!(vma->vm_flags & VM_WRITE)`), 则判定为非法访问, 直接返回错误。

2. 按需分页 (Demand Paging) 支持:

- 如果页表项 (`*ptep`) 全为 0, 说明该页面尚未分配物理内存。
- 函数调用 `pgdir_alloc_page` 分配物理页并建立映射, 使得用户程序可以动态使用内存 (例如栈空间的自动增长)。

3. 写时复制 (Copy-on-Write) 支持 (Challenge 实现):

- 当检测到写操作访问一个硬件标记为只读（!PTE_W）但 VMA 逻辑上标记为可写（VM_WRITE）的页面时，系统判断触发了 COW 机制。我们根据物理页的引用计数（page_ref）分情况处理：
- **情况 A：共享页（page_ref > 1）**
 - **场景：**父进程和子进程（fork 产生）共享同一个物理页。
 - **操作：**
 - a. alloc_page() 分配一个新的物理页。
 - b. memcpy() 将旧页面的内容完整拷贝到新页。
 - c. page_insert() 将虚拟地址映射到新页，并赋予 PTE_W (可写) 权限。
 - **结果：**原页面的引用计数减少，新页面属于当前进程私有。
- **情况 B：独占页（page_ref == 1）**
 - **场景：**虽然该页在页表中是只读的，但引用计数为 1，说明其他共享该页的进程已经退出了，或者是多次 COW 后剩下的最后一个持有者。
 - **操作：**
 - a. 无需分配新内存。
 - b. 直接通过 page_insert 重新映射该物理页，但在权限中加上 PTE_W，恢复其可写状态。

通过添加 do_pgfault 并正确挂载到 trap.c，uCore 具备了处理复杂内存异常的能力。这不仅支持了用户程序的按需内存分配，更通过 COW 机制极大地优化了进程创建（fork）时的性能和内存开销。

添加之后，make grade 的测试时间大幅减少，在测试 faultread、faultreadkernel 和 testbss 等用例时，执行时间和其他测试时间一样。

```

faultread:                (1.3s)
-check result:             OK
-check output:             OK
faultreadkernel:          (1.3s)
-check result:             OK
-check output:             OK
hello:                    (1.2s)
-check result:             OK
-check output:             OK
testbss:                  (1.3s)
-check result:             OK
-check output:             OK

```

do_fork函数修改

```

int do_fork(uint32_t clone_flags, uintptr_t stack, struct trapframe *tf)
{
    int ret = -E_NO_FREE_PROC;
    struct proc_struct *proc;
    if (nr_process >= MAX_PROCESS)

```

```

{
    goto fork_out;
}
ret = -E_NO_MEM;

if ((proc = alloc_proc()) == NULL) {
    goto fork_out;
}

// 设置子进程的父进程为当前进程
proc->parent = current;

// 确保当前进程的 wait_state 为 0 (确保父进程状态正常)
assert(current->wait_state == 0);

// 2. 调用 setup_kstack 为子进程分配内核栈
if (setup_kstack(proc) != 0) {
    goto bad_fork_cleanup_proc;
}

// 3. 调用 copy_mm 复制或共享内存管理结构
if (copy_mm(clone_flags, proc) != 0) {
    goto bad_fork_cleanup_kstack;
}

// 4. 调用 copy_thread 设置 trapframe 和 context
copy_thread(proc, stack, tf);

// 5. 将新进程插入 hash_list 和 proc_list, 并设置进程关系链接
// 为了防止在操作链表时发生中断导致竞争条件, 最好关闭中断
bool intr_flag;
local_intr_save(intr_flag);
{
    proc->pid = get_pid(); // 获取唯一的 PID
    hash_proc(proc);      // 建立 hash 映射
    set_links(proc);      // 设置进程链接 (加入 proc_list, 维护父子兄弟链表, nr_process++)
}
local_intr_restore(intr_flag);

// 6. 唤醒新进程
wakeup_proc(proc);

// 7. 返回新进程的 PID

```

```

    ret = proc->pid;
fork_out:
    return ret;

bad_fork_cleanup_kstack:
    put_kstack(proc);
bad_fork_cleanup_proc:
    kfree(proc);
    goto fork_out;
}

```

在之前的基础上添加 `proc->parent = current`，将当前的进程设置为子进程的父进程。`assert(current->wait_state == 0)`确保了当前进程的等待状态为0。`set_links(proc)`设置进程链接(加入 `proc_list`, 维护父子兄弟链表, `nr_process++`)

interrupt_handler函数修改

```

case IRQ_S_TIMER:
    // (1) 设置下次时钟中断 (保持心跳)
    clock_set_next_event();

    // (2) 计数器 (ticks) 加一 (更新系统时间

    // (3) 检查时间片是否耗尽
    ticks++;
    if (current != NULL) {
        if (current->time_slice > 0) {
            current->time_slice--;
        }

        // 当时间片耗尽时，标记需要调度
        if (current->time_slice <= 0) {
            current->need_resched = 1;
        }
    }
    break;

```

更新case `IRQ_S_TIMER`，实现时间片功能轮转函数，设置下一次时钟中断 (`clock_set_next_event`)，`ticks` 计数器自增，每 `TICK_NUM` 次中断 (100 次)，进行判断当前是否有进程正在运行，如果有则标记该进程需要被重新调度 (`current->need_resched`)


```

        // 或者它刚被创建/唤醒。我们需要给它分配新的时间片，才能让它运行。
        if (next->time_slice == 0) {
            next->time_slice = 3; // 重置时间片为 3 个 tick
        }
        break; // 找到目标进程，跳出循环
    }
}

} while (le != last);

// 如果未找到可运行进程，或者找到的进程状态不对，则运行 idleproc
if (next == NULL || next->state != PROC_RUNNABLE)
{
    next = idleproc;
}

next->runs++;
if (next != current)
{
    proc_run(next);
}
}
local_intr_restore(intr_flag);
}

```

整个调度系统通过 `trap.c` 和 `sched.c` 的协同工作完成闭环：

1. **消耗 (trap.c)**：在时钟中断中，当前运行进程的 `time_slice` 逐步减少。当减为 0 时，`trap.c` 设置 `need_resched = 1`，并在中断返回前触发 `schedule()`。
2. **重置 (sched.c)**：`schedule()` 被调用，它遍历队列找到这个刚被暂停的（或者其他就绪的）进程。发现其 `time_slice` 为 0，于是将其重置为初始值（代码中硬编码为 3）。
3. **执行 (proc_run)**：调用 `proc_run(next)` 完成上下文切换，CPU 开始执行目标进程，直到其时间片再次耗尽或主动放弃 CPU。

这种改造实现了：

- **抢占性**：通过在时钟中断中检查 `time_slice`，内核能够强行剥夺超时进程的 CPU 使用权。
- **公平性**：保证了所有 `RUNNABLE` 的进程都有机会获得 CPU 资源，防止某个死循环的用户进程卡死整个系统。

alloc_proc函数

```
static struct proc_struct *
alloc_proc(void)
{
    struct proc_struct *proc = kmalloc(sizeof(struct proc_struct));
    if (proc != NULL)
    {
        // LAB5 YOUR CODE : 2312220(update LAB4 steps)
        /*
         * below fields(add in LAB5) in proc_struct need to be initialized
         *      uint32_t wait_state;                // waiting state
         *      struct proc_struct *cptr, *yptr, *optr;    // relations between processes
         */
        proc->wait_state = 0; // 初始化等待状态为 0 (无等待)
        // 初始化进程关系链表指针为 NULL
        // cptr: 指向最年轻的子进程 (Child Pointer)
        // yptr: 指向下一个更年轻的兄弟进程 (Younger Sibling Pointer)
        // optr: 指向下一个更年长的兄弟进程 (Older Sibling Pointer)
        proc->cptr = proc->optr = proc->yptr = NULL;
        proc->time_slice = 0;
    }
    return proc;
}
```

在已有的函数上，添加 wait_state、*cptr, *yptr, *optr的初始化操作，以及引入的时间片time_slice的初始化操作。

练习1: 加载应用程序并执行（需要编码）

trapframe代码

问题

do_exec函数调用 load_icode（位于 kern/process/proc.c中）来加载并解析一个处于内存中的ELF执行文件格式的应用程序。你需要补充 load_icode的第6步，建立相应的用户内存空间来放置应用程序的代码段、数据段等，且要设置好 proc_struct结构中的成员变量 trapframe中的内容，确保在执行此进程后，能够从应用程序设定的起始执行地址开始执行。需设置正确的 trapframe内容。

回答

代码如下所示：

```
// 1. 设置用户栈指针
// USTACKTOP 是用户态栈的顶部地址，定义在 memlayout.h 中
// 用户程序开始执行时，sp 寄存器必须指向合法的用户栈
tf->gpr.sp = USTACKTOP;
// 2. 设置程序的入口地址
// elf->e_entry 存储了 ELF 文件的入口地址（_start）
// 当 sret 指令执行后，CPU 会跳转到 sepc 寄存器指向的地址，即这里的 e_entry
tf->epc = elf->e_entry;
// 3. 设置状态寄存器 (sstatus)
// 这是特权级切换的关键：
// (1) read_csr(ssstatus): 读取当前 sstatus 的值
// (2) & ~SSTATUS_SPP: 清除 SPP (Supervisor Previous Privilege) 位。
//     SPP=1 表示之前是 S 态，SPP=0 表示之前是 U 态。
//     我们要让 CPU 认为“我是从用户态进入中断的”，这样 sret 时就会“返回”到用户态。
// (3) | SSTATUS_SPIE: 设置 SPIE (Supervisor Previous Interrupt Enable) 位。
//     确保回到用户态后，中断是开启的 (Interrupt Enabled)，否则操作系统将失去对 CPU 的控制权。
tf->status = (read_csr(ssstatus) & ~SSTATUS_SPP) | SSTATUS_SPIE;
```

我们将 sp 寄存器设置为栈顶 USTACKTOP，然后将 epc 寄存器设置为文件的入口地址，将 sstatus 的 SPP 位清零，表示异常来自用户态；同时将 sstatus 的 SPIE 位清零，启用中断。

用户态进程启动执行流程

从内核线程被调度运行，到加载应用程序并执行其第一条指令，整个过程可以概括为以下几个阶段：

1. 内核线程启动与伪装

阶段描述：系统首先在内核态创建一个内核线程，该线程作为“父进程”的替身，准备发起加载应用程序的请求。

- **创建进程：**在 `init_main` 中，通过 `kernel_thread(user_main, NULL, 0)` 调用 `do_fork` 创建并唤醒线程。
- **开始运行：**调度器选择该线程，状态变为 `PROC_RUNNABLE`，最终跳转到 `user_main` 函数执行。

2. 发起系统调用 (System Call)

阶段描述：`user_main` 函数利用内联汇编或宏，通过中断/异常机制陷入内核，请求执行新的程序。

- **触发异常：**在 `user_main` 中调用 `KERNEL_EXECVE(exit)` (实际宏展开调用 `kern_execve`)。

- **断点指令：** kern_execve 内部执行 ebreak 指令，触发断点异常。
- **异常分发：**
 - i. CPU 跳转到 __alltraps 保存上下文。
 - ii. 调用 trap -> trap_dispatch -> exception_handler。
 - iii. 识别到 CAUSE_BREAKPOINT，最终调用 syscall 函数。

3. 执行加载逻辑 (Loading Binary)

阶段描述：内核处理系统调用，读取应用程序二进制文件，并构建用户态的运行环境。

- **系统调用分发：**

在 syscall 函数中，根据 tf->gpr.a0 (系统调用号) 找到对应的处理函数。

```
// 此时 num 对应 SYS_exec
tf->gpr.a0 = syscalls[num](arg);
```

- **执行 Exec：**调用路径为 sys_exec -> do_execve。
- **加载代码 (关键步骤)：**

do_execve 调用 load_icode(binary, size)。它完成了以下工作：

 - i. **内存空间：**建立新的页表，分配用户内存空间。
 - ii. **内容拷贝：**解析 ELF 格式，将程序内容加载到内存。
 - iii. **TrapFrame 修改：这是控制流转移的关键。**它将当前进程的中断帧 (tf) 中的 epc (Exception Program Counter) 设置为应用程序的入口地址 (Entry Point)，并将 sp 设置为用户栈顶，同时将状态寄存器设置为用户态 (User Mode)。

4. 返回用户态与执行第一条指令

阶段描述：内核完成工作，通过从中断返回指令，利用修改后的 TrapFrame 将 CPU 切换到用户态并跳转到程序入口。

- **逐层返回：**load_icode 返回 -> do_execve 返回 -> sys_exec 返回 -> syscall 返回 -> trap 返回。
- **恢复上下文：**

执行流回到 __alltraps 的末尾，执行 __trapret。该汇编代码段会将之前保存在栈上的 TrapFrame 内容恢复到 CPU 寄存器中。
- **特权级切换 (sret)：**

最后执行 sret (Supervisor Return) 指令。硬件会进行如下操作：

 - i. **跳转：**将 PC (程序计数器) 设置为 sepc 寄存器的值 (此前 load_icode 已将其设为**应用程序第一条指令的地址**)。
 - ii. **切换模式：**根据 sstatus 寄存器的 SPP 位，将 CPU 特权级从 S 态 (内核态) 切换回 U 态 (用户态)。

最终结果：CPU 处于用户态，PC 指向应用程序入口，开始执行程序的第一条指令。

练习2: 父进程复制自己的内存空间给子进程（需要编码）

copy_range

问题

创建子进程的函数 `do_fork` 在执行中将拷贝当前进程（即父进程）的用户内存地址空间中的合法内容到新进程中（子进程），完成内存资源的复制。具体是通过 `copy_range` 函数（位于 `kern/mm/pmm.c` 中）实现的，请补充 `copy_range` 的实现，确保能够正确执行。

回答

代码如下所示：

```
int copy_range(pde_t *to, pde_t *from, uintptr_t start, uintptr_t end, bool share)
{
    assert(start % PGSIZE == 0 && end % PGSIZE == 0);
    assert(USER_ACCESS(start, end));
    // copy content by page unit.
    do
    {
        // call get_pte to find process A's pte according to the addr start
        pte_t *ptep = get_pte(from, start, 0), *nptep;
        if (ptep == NULL)
        {
            start = ROUNDDOWN(start + PTSIZE, PTSIZE);
            continue;
        }
        // call get_pte to find process B's pte according to the addr start. If
        // pte is NULL, just alloc a PT
        if (*ptep & PTE_V)
        {
            if ((nptep = get_pte(to, start, 1)) == NULL)
            {
                return -E_NO_MEM;
            }
            uint32_t perm = (*ptep & PTE_USER);
            // get page from ptep
            struct Page *page = pte2page(*ptep);
            int ret = 0;
```

```

// LAB5 CHALLENGE: Copy on Write 逻辑
if (share)
{
    // 如果页面是可写的，需要将其标记为只读，以便父子进程共享
    if (perm & PTE_W) {
        perm &= ~PTE_W;
        // 修改父进程的映射：去掉写权限
        // page_insert 会自动处理引用计数(ref不变)并刷新TLB
        ret = page_insert(from, page, start, perm);
        if (ret != 0) return ret;
    }

    // 将该页面映射给子进程，权限与父进程一致（只读）
    // page_insert 会自动增加物理页的引用计数 (ref++)
    ret = page_insert(to, page, start, perm);
    assert(ret == 0);
}
else
{
    // --- 原有逻辑 (Deep Copy) ---
    // alloc a page for process B
    // 我们将分配移到了这里，只有在非共享模式下才分配新页
    struct Page *npage = alloc_page();
    assert(page != NULL);
    assert(npage != NULL);

    /* LAB5:EXERCISE2 2312220
    */
    // 1. 获取源页面的内核虚拟地址
    void *kva_src = page2kva(page);
    // 2. 获取目标页面（新分配页）的内核虚拟地址
    void *kva_dst = page2kva(npage);
    // 3. 复制内存内容（复制一整页 4096 字节）
    memcpy(kva_dst, kva_src, PGSIZE);
    // 4. 建立物理地址与线性地址的映射
    ret = page_insert(to, npage, start, perm);
    assert(ret == 0);
}
}
start += PGSIZE;
} while (start != 0 && start < end);
return 0;

```

```
}
```

具体的实现：

1. 首先获取 `src` 源地址和 `dst` 目的地址的内核虚拟地址；
2. 拷贝内存，将 `src` 的内存复制到 `dst` 中；
3. 最后将拷贝完的页插入到页表中即可,建立物理地址与线性地址的映射.

Copy on Write 机制设计与实现

1. 概要设计 (High-Level Design)

COW 的核心思想是**推迟物理内存的复制**，直到真正发生写操作时才进行。设计分为两个关键阶段：

- **进程创建阶段 (Fork)：**
在 `fork` 时，不直接复制父进程的物理内存。而是将父进程和子进程的虚拟页表项 (PTE) 指向**同一个物理页**，并将双方的 PTE 权限都设置为**只读 (Read-Only)**。同时，增加该物理页的引用计数。
- **运行时异常处理阶段 (Page Fault)：**
当任一进程尝试写入这个只读的共享页时，CPU 触发缺页异常。内核捕获该异常，检测到是 COW 行为，则分配新的物理页，将数据复制过去，并更新页表项为**可写 (Writable)**，从而实现私有化。

2. 详细设计 (Detailed Design)

详细实现需要修改内存管理模块 (`pmm.c`) 和异常处理模块 (`vmm.c`)，并维护物理页的引用计数。

A. 数据结构与状态机

利用物理页结构 `struct Page` 中的 `ref` (引用计数) 来管理页面状态：

- **共享只读态** (`ref > 1`)：多个进程共享，PTE 无写权限 (`!PTE_W`)。
- **私有可写态** (`ref == 1`)：仅当前进程独占，PTE 有写权限 (`PTE_W`)。

B. 修改 Fork 逻辑 (`kern/mm/pmm.c`)

在 `copy_range` 函数中，当 `share` 标志开启时，执行浅拷贝：

1. **遍历页表**：找到父进程的 PTE。
2. **降级权限**：如果父进程页面是可写的 (`PTE_W`)，清除父进程 PTE 的写权限位，刷新 TLB。
3. **共享映射**：将子进程的 PTE 映射到同一个物理页，权限同样设为只读。
4. **增加引用**：`page_insert` 调用会自动增加物理页的引用计数 (`page->ref++`)。

```
// 伪代码逻辑
if (share) {
    if (perm & PTE_W) {
```

```

    perm &= ~PTE_W; // 清除写权限
    page_insert(from, page, start, perm); // 更新父进程 PTE
}
page_insert(to, page, start, perm); // 映射给子进程, ref++
}

```

C. 修改缺页异常逻辑 (kern/mm/vmm.c)

在 do_pgfault 函数中增加对 COW 的判定和处理：

1. 判定条件：

- 异常由写操作引起 (error_code & 2)。
- 页表项存在 (PTE_V) 但不可写 (!PTE_W)。
- 虚拟内存区域 (VMA) 标记为可写 (VM_WRITE)。

2. 处理流程：

- **情况 1：多进程共享 (page->ref > 1)**
 - 申请新物理页 (alloc_page)。
 - 拷贝原页面内容 (memcpy)。
 - 建立新映射：将当前虚拟地址映射到新页，并设置**可写权限** (PTE_W)。原物理页引用计数自动减 1。
- **情况 2：单进程独占 (page->ref == 1)**
 - 无需拷贝。
 - 直接修改当前 PTE，增加**可写权限** (PTE_W)。
 - 刷新 TLB。

// 伪代码逻辑

```

if ((error_code & WRITE) && (pte_val & PTE_V) && !(pte_val & PTE_W)) {
    if (page->ref > 1) {
        // 还有其他进程共享，必须复制
        new_page = alloc_page();
        memcpy(new_page, old_page, PGSIZE);
        page_insert(mm->pgdir, new_page, addr, perm | PTE_W);
    } else {
        // 只有当前进程在使用，直接恢复写权限
        page_insert(mm->pgdir, old_page, addr, perm | PTE_W);
    }
}

```


练习3: 阅读分析源代码，理解进程执行 fork/exec/wait/exit 的实现，以及系统调用的实现

一、 源代码详细分析

1. fork 函数分析：创建新进程

功能描述：

fork 通过复制父进程的内存和上下文来创建一个新的子进程。在 uCore 中， sys_fork 只是一个包装，实际逻辑在 do_fork 中完成。

代码详细注释与分析：

```
// sys_fork: 系统调用入口，获取当前进程的中断帧和栈指针
static int
sys_fork(uint64_t arg[]) {
    struct trapframe *tf = current->tf; // 获取当前进程的中断帧
    uintptr_t stack = tf->gpr.sp;         // 获取当前栈指针
    return do_fork(0, stack, tf);         // 调用实际的 fork 处理函数
}

// do_fork: 创建进程的核心实现
int
do_fork(uint32_t clone_flags, uintptr_t stack, struct trapframe *tf) {
    int ret = -E_NO_FREE_PROC;
    struct proc_struct *proc;

    // 1. 检查当前进程数是否已达上限
    if (nr_process >= MAX_PROCESS) {
        goto fork_out;
    }
    ret = -E_NO_MEM;

    // 2. 分配并初始化进程控制块 (PCB)
    // alloc_proc 会负责分配内存并将 proc_struct 字段清零初始化
    if((proc = alloc_proc()) == NULL) {
        goto fork_out;
    }

    // 3. 建立父子关系
    proc->parent = current; // 将子进程的父进程设置为当前进程
```

```

// 确保当前进程不在等待状态，这是 fork 的前提
assert(current->wait_state == 0);

// 4. 分配内核栈
// 每个进程都需要独立的内核栈来保存中断帧和内核执行上下文
if(setup_kstack(proc) != 0) {
    goto bad_fork_cleanup_proc;
}

// 5. 复制或共享内存空间 (copy_mm)
// 根据 clone_flags 决定是复制页表(Deep Copy/COW)还是共享内存(Threads)
// 对于 fork，通常是复制页表（配合 COW 技术）
if(copy_mm(clone_flags, proc) != 0) {
    goto bad_fork_cleanup_kstack;
}

// 6. 复制线程上下文 (copy_thread)
// 关键步骤：设置子进程的 trapframe 和 context。
// 这使得子进程被调度时，能从 fork 调用处“返回”，且返回值为 0。
copy_thread(proc, stack, tf);

// 7. 将新进程加入系统管理（加锁保护）
bool intr_flag;
local_intr_save(intr_flag); // 关中断，保证原子性
{
    int pid = get_pid(); // 获取唯一的 PID
    proc->pid = pid;
    hash_proc(proc); // 插入进程哈希表，便于通过 PID 查找
    set_links(proc); // 插入进程链表，便于遍历
}
local_intr_restore(intr_flag); // 开中断

// 8. 唤醒子进程
// 将子进程状态设为 PROC_RUNNABLE，使其可以被调度器选中运行
wakeup_proc(proc);

// 9. 返回子进程的 PID 给父进程
ret = proc->pid;

fork_out:
    return ret;

```

```
// 异常处理：回滚操作，释放已分配的资源
bad_fork_cleanup_kstack:
    put_kstack(proc);
bad_fork_cleanup_proc:
    kfree(proc);
    goto fork_out;
}
```

2. exec 函数分析：加载并执行新程序

功能描述：

exec 用于回收当前进程的内存空间，并加载新的二进制程序（ELF格式）来覆盖当前进程，从而执行新的逻辑。

代码详细注释与分析：

```
// sys_exec: 系统调用入口，提取参数
static int
sys_exec(uint64_t arg[]) {
    const char *name = (const char *)arg[0];
    size_t len = (size_t)arg[1];
    unsigned char *binary = (unsigned char *)arg[2];
    size_t size = (size_t)arg[3];
    return do_execve(name, len, binary, size);
}

// do_execve: 加载新程序的核心实现
int
do_execve(const char *name, size_t len, unsigned char *binary, size_t size) {
    struct mm_struct *mm = current->mm;

    // 1. 检查用户传入的内存地址是否合法
    if (!user_mem_check(mm, (uintptr_t)name, len, 0)) {
        return -E_INVALID;
    }
    if (len > PROC_NAME_LEN) {
        len = PROC_NAME_LEN;
    }
}
```

```

// 2. 暂存新程序的名称
char local_name[PROC_NAME_LEN + 1];
memset(local_name, 0, sizeof(local_name));
memcpy(local_name, name, len);

// 3. 回收当前进程的旧内存空间
if (mm != NULL) {
    cputs("mm != NULL");
    lcr3(boot_cr3); // 切换回内核页表，因为当前用户页表即将被销毁

    // 如果引用计数减为0，说明没有其他线程共享此内存，彻底释放
    if (mm_count_dec(mm) == 0) {
        exit_mmap(mm); // 释放用户空间的物理页映射
        put_pgdir(mm); // 释放页目录表
        mm_destroy(mm); // 销毁 mm 结构体
    }
    current->mm = NULL; // 解除当前进程与旧 mm 的关联
}

// 4. 加载新程序的二进制代码（核心步骤）
// load_icode 会：
// a. 解析 ELF header
// b. 建立新的 mm_struct 和页表
// c. 将代码段/数据段加载到内存
// d. 建立用户栈
// e. 修改当前进程的中断帧 (tf)，将 PC 设为新程序的入口地址
int ret;
if ((ret = load_icode(binary, size)) != 0) {
    goto execve_exit; // 加载失败
}

// 5. 设置进程新名称
set_proc_name(current, local_name);
return 0; // 成功返回（实际上返回用户态时会跳转到新程序入口）

execve_exit:
// 如果加载失败，进程状态已经破坏，只能退出
do_exit(ret);
panic("already exit: %e.\n", ret);
}

```

3. wait 函数分析：等待子进程结束

功能描述：

父进程调用 wait 等待子进程退出。如果子进程已经退出（ZOMBIE），则回收其资源；如果子进程还在运行，父进程进入睡眠状态。

代码详细注释与分析：

```
// sys_wait: 系统调用入口
static int
sys_wait(uint64_t arg[]) {
    int pid = (int)arg[0];
    int *store = (int *)arg[1];
    return do_wait(pid, store);
}

// do_wait: 等待子进程的核心实现
int
do_wait(int pid, int *code_store) {
    struct mm_struct *mm = current->mm;
    // 1. 检查用于存储退出码的地址是否合法
    if (code_store != NULL) {
        if (!user_mem_check(mm, (uintptr_t)code_store, sizeof(int), 1)) {
            return -E_INVAL;
        }
    }

    struct proc_struct *proc;
    bool intr_flag, haskid;

repeat: // 循环点：如果被唤醒但不是目标子进程退出，需要再次检查
    haskid = 0;

    // 2. 查找目标子进程
    if (pid != 0) {
        // 等待特定的 pid
        proc = find_proc(pid);
        if (proc != NULL && proc->parent == current) {
            haskid = 1;
            // 如果子进程已经是僵尸状态，直接跳转到回收逻辑
            if (proc->state == PROC_ZOMBIE) {
                goto found;
            }
        }
    }
```

```

    }
}
else {
    // pid == 0, 等待任意一个子进程
    proc = current->cptr; // 获取第一个子进程
    for (; proc != NULL; proc = proc->optr) {
        haskid = 1;
        if (proc->state == PROC_ZOMBIE) {
            goto found;
        }
    }
}
}

```

// 3. 处理等待逻辑

```

if (haskid) {
    // 如果有子进程但都没退出，当前进程进入睡眠
    current->state = PROC_SLEEPING;
    current->wait_state = WT_CHILD; // 标记等待原因为“等待子进程”
    schedule(); // 让出 CPU，触发调度

    // 当被唤醒后（通常是 do_exit 唤醒），从这里继续执行
    if (current->flags & PF_EXITING) {
        do_exit(-E_KILLED);
    }
    goto repeat; // 再次检查是否有子进程变为僵尸
}

```

// 如果没有找到符合条件的子进程

```

return -E_BAD_PROC;

```

found:

```

// 4. 回收僵尸进程资源
if (proc == idleproc || proc == initproc) {
    panic("wait idleproc or initproc.\n");
}

```

// 保存子进程的退出码

```

if (code_store != NULL) {
    *code_store = proc->exit_code;
}

```

// 从哈希表和进程链表中移除子进程

```

local_intr_save(intr_flag);

```

```

{
    unhash_proc(proc);
    remove_links(proc);
}
local_intr_restore(intr_flag);

// 彻底释放子进程的内核栈和 PCB
put_kstack(proc);
kfree(proc);
return 0;
}

```

4. exit 函数分析：退出进程

功能描述：

进程主动结束运行，释放大部分资源，变为 ZOMBIE 状态，并通知父进程来回收剩余资源（PCB和内核栈）。

代码详细注释与分析：

```

// sys_exit: 系统调用入口
static int
sys_exit(uint64_t arg[]) {
    int error_code = (int)arg[0];
    return do_exit(error_code);
}

// do_exit: 进程退出的核心实现
int
do_exit(int error_code) {
    // 0号和1号进程不允许退出
    if (current == idleproc) {
        panic("idleproc exit.\n");
    }
    if (current == initproc) {
        panic("initproc exit.\n");
    }

    // 1. 释放虚拟内存空间
    struct mm_struct *mm = current->mm;

```

```

if (mm != NULL) {
    lcr3(boot_cr3); // 切回内核页表
    if (mm_count_dec(mm) == 0) {
        exit_mmap(mm);
        put_pgdir(mm);
        mm_destroy(mm);
    }
    current->mm = NULL;
}

// 2. 设置进程状态为僵尸 (ZOMBIE)
current->state = PROC_ZOMBIE;
current->exit_code = error_code; // 记录退出码

bool intr_flag;
struct proc_struct *proc;

local_intr_save(intr_flag);
{
    // 3. 唤醒父进程
    proc = current->parent;
    if (proc->wait_state == WT_CHILD) {
        wakeup_proc(proc); // 如果父进程在 wait, 唤醒它
    }

    // 4. 处理孤儿进程 (子进程过继)
    // 当前进程退出了, 它的子进程需要找新的父亲 (通常是 initproc)
    while (current->cptr != NULL) {
        proc = current->cptr;
        current->cptr = proc->optr; // 从当前进程的子进程链表中移除

        proc->yptr = NULL;
        // 将子进程插入 initproc 的子进程链表
        if ((proc->optr = initproc->cptr) != NULL) {
            initproc->cptr->yptr = proc;
        }
        proc->parent = initproc; // 设置父进程为 initproc
        initproc->cptr = proc;

        // 如果被过继的子进程已经是僵尸, 需要通知新父亲(initproc)来回收
        if (proc->state == PROC_ZOMBIE) {
            if (initproc->wait_state == WT_CHILD) {
                wakeup_proc(initproc);
            }
        }
    }
}

```



```

    }
}
}
local_intr_restore(intr_flag);

// 5. 触发调度
// 因为当前进程已变成 ZOMBIE，无法继续运行，必须切换到其他进程
schedule();

// 理论上永远不会执行到这里
panic("do_exit will not return!! %d.\n", current->pid);
}

```

二、执行流程与机制问答

1. fork/exec/wait/exit 的执行流程分析

- 用户态与内核态的操作区分：

- **用户态**：用户程序调用标准库封装的函数（如 `fork()`）。这些函数仅仅是发起者，它们准备好参数，然后执行特权指令（如 RISC-V 的 `ecall` 或 x86 的 `int 0x80`）来触发异常，从而陷入内核。
- **内核态**：所有的实质性资源管理工作都在内核态完成。
 - `fork`：在内核态分配 `proc_struct`，分配内核栈，复制页表。
 - `exec`：在内核态读取 ELF 文件，清空旧内存映射，建立新映射。
 - `wait`：在内核态检查进程链表，修改进程调度状态（SLEEPING），执行上下文切换。
 - `exit`：在内核态释放内存页表，重组父子进程关系树。

- 内核态与用户态程序的交错执行：

- 执行流并非连续的单一线性流，而是通过 **中断/异常 (Trap)** 机制在特权级之间跳转。
- **进入内核**：用户程序执行系统调用指令 -> 触发异常 -> CPU 跳转到 `__alltraps` -> 保存用户态寄存器到 `TrapFrame` -> 调用 `trap` -> `trap_dispatch` -> `syscall` -> `sys_*` -> `do_*`。
- **回到用户**：内核处理完毕 -> `__trapret` (恢复 `TrapFrame` 中的寄存器) -> 执行 `sret` 指令 -> CPU 恢复状态并跳转回用户程序（PC指向系统调用指令的下一条，或新程序的入口）。

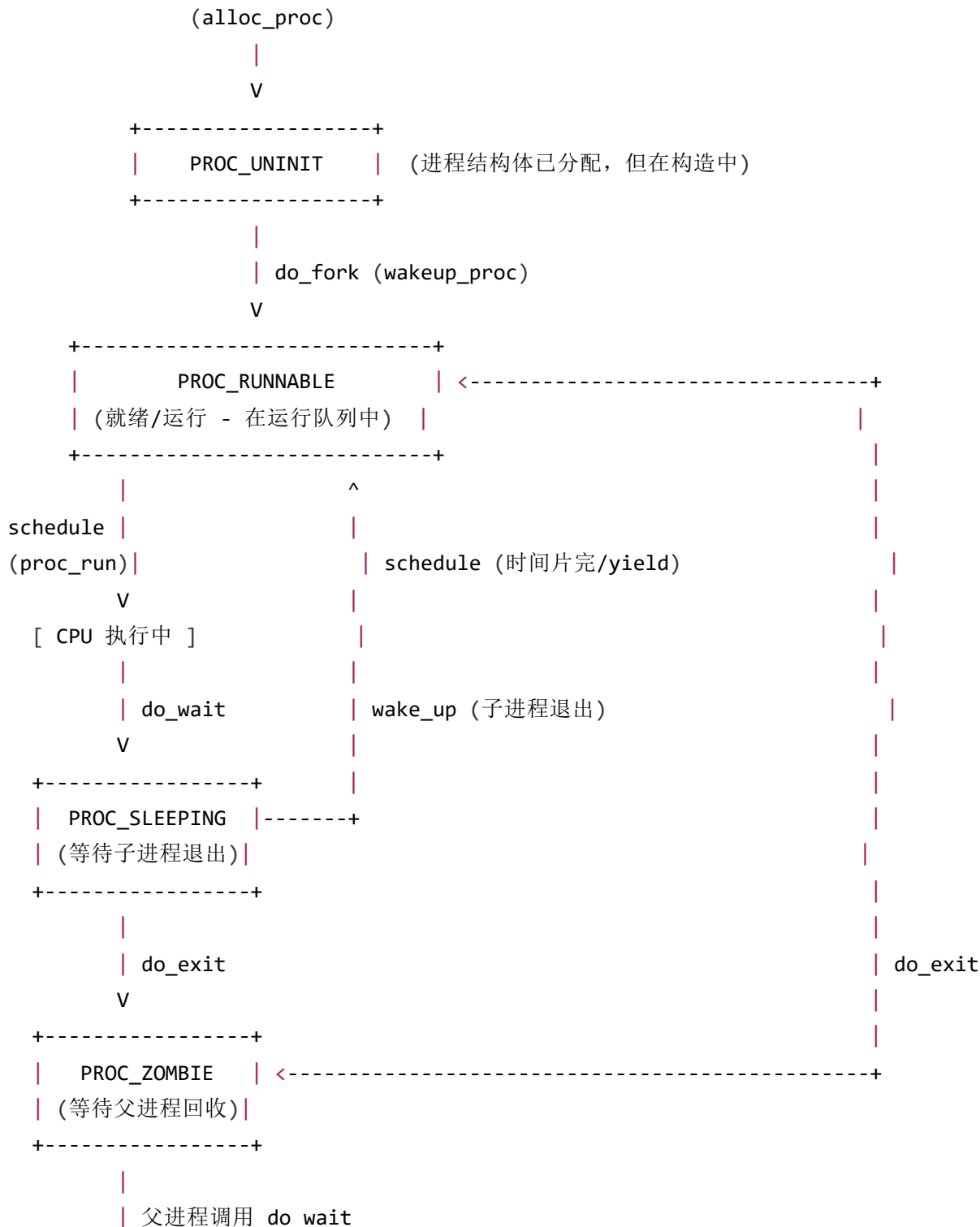
- 内核态执行结果如何返回给用户程序：

- 结果通过 **TrapFrame (中断帧)** 返回。
- 在 `syscall` 函数中：`tf->gpr.a0 = syscalls[num](arg);`
- 内核将核心处理函数（如 `do_fork`）的返回值写入当前进程中断帧的 `a0` 寄存器位置。

- 当执行 `sret` 返回用户态时，`__trapret` 汇编代码会将 `TrapFrame` 中的值恢复到物理寄存器，因此用户程序读取 `a0` 寄存器就得到了系统调用的返回值（例如 `fork` 返回的子进程 PID）。

三、用户态进程执行状态生命周期图

根据 uCore 的调度逻辑和提供的资料，进程生命周期及转换关系如下：



V

(资源彻底释放)

(kfree proc)

状态流转说明:

1. **UNINIT -> RUNNABLE**: do_fork 完成进程初始化后, 调用 wakeup_proc 将其设为 RUNNABLE。
2. **RUNNABLE <-> RUNNING**: 在 uCore 实现中, PROC_RUNNABLE 涵盖了就绪和运行。调度器 schedule 选中一个 RUNNABLE 进程上 CPU 执行 (逻辑上的 RUNNING); 当时间片用完或调用 sys_yield, 进程被放回队列, 状态仍为 RUNNABLE (或重新标记为 RUNNABLE)。
3. **RUNNING -> SLEEPING**: 进程调用 do_wait 且目标子进程未退出, 状态变为 PROC_SLEEPING 并让出 CPU。
4. **SLEEPING -> RUNNABLE**: 当子进程执行 do_exit 时, 会扫描父进程, 如果父进程在等待 (WT_CHILD), 则调用 wakeup_proc 唤醒父进程。
5. **RUNNING/SLEEPING -> ZOMBIE**: 进程调用 do_exit (或被 kill), 释放大部分资源, 变成僵尸状态, 等待父进程回收 PCB。

测试点全部通过

```
badsegment:                (1.2s)
  -check result:            OK
  -check output:            OK
divzero:                    (1.2s)
  -check result:            OK
  -check output:            OK
softint:                    (1.2s)
  -check result:            OK
  -check output:            OK
faultread:                  (1.3s)
  -check result:            OK
  -check output:            OK
faultreadkernel:            (1.3s)
  -check result:            OK
  -check output:            OK
hello:                      (1.2s)
  -check result:            OK
  -check output:            OK
testbss:                    (1.3s)
  -check result:            OK
  -check output:            OK
pgdir:                      (1.2s)
  -check result:            OK
  -check output:            OK
yield:                      (1.2s)
  -check result:            OK
  -check output:            OK
badarg:                     (1.2s)
  -check result:            OK
  -check output:            OK
exit:                       (1.2s)
  -check result:            OK
  -check output:            OK
spin:                       (1.8s)
  -check result:            OK
  -check output:            OK
forktest:                   (1.2s)
  -check result:            OK
  -check output:            OK
Total Score: 130/130
xwy@DESKTOP-JUDTUK3:/mnt/d/OS/lab5(1)/1$
```

Challenge1: Copy-on-Write (COW) 设计与实现报告

一、设计思路与原理

1.1 核心机制

Copy-on-Write (COW) 是一种内存管理优化技术。在 ucore 中, 标准的 `fork()` 会调用 `copy_range` 将父进程的内存空间完全复制一份给子进程 (Deep Copy), 这在内存占用和时间开销上都是巨大的浪费, 特别是当子进程 `fork` 后立即执行 `exec` 时。

COW 的改进策略是:

1. **Fork 阶段**: 不复制物理内存。父子进程共享同一个物理页, 但将双方的页表项 (PTE) 都标记为 **只读 (Read-Only)**。
2. **运行阶段**: 当任一进程尝试写入该页时, CPU 触发 **缺页异常 (Page Fault)**。
3. **异常处理**: 内核捕获异常, 检测到是 COW 引起的写入冲突, 分配新的物理页, 将数据复制过去, 并更新页表项为 **可写 (Writable)**, 从而实现“按需复制”。

1.2 有限状态自动机 (FSM) 分析

物理页面的状态流转可以用以下有限状态自动机描述:

- **状态 A: 私有可写 (Private / Writable)**
 - **特征**: `page->ref == 1`, 页表项权限包含 `PTE_W`。
 - **含义**: 页面仅被一个进程独占, 可以自由读写。
 - **场景**: 进程刚申请内存, 或 COW 发生后。
- **状态 B: 共享只读 (Shared / Read-Only)**
 - **特征**: `page->ref > 1`, 所有映射该页的 PTE 均无 `PTE_W`。
 - **含义**: 页面被父子进程共享, 任何一方写入都会触发异常。
 - **场景**: `fork()` 执行完毕后。
- **状态转换事件**:
 - i. **Fork (A -> B)**:
 - 父进程创建子进程。
 - 操作: 将父进程 PTE 的 `PTE_W` 清除, `page->ref++`, 子进程映射同一物理页且无 `PTE_W`。
 - ii. **Write Fault (B -> A) [核心转换]**:
 - 进程尝试写入状态 B 的页面。
 - **分支 1 (Copy)**: 若 `ref > 1`。分配新页 `P'`, 复制 `P` 内容至 `P'`。当前进程映射 `P'` (`PTE_W=1`), 原 `P` 引用计数减 1。
 - **分支 2 (Reuse)**: 若 `ref == 1` (例如其他共享进程已退出)。无需复制, 直接恢复当前 PTE 的 `PTE_W`。

二、源代码实现

2.1 修改 kern/mm/pmm.c

重构 copy_range 函数，实现共享映射逻辑。

```
int copy_range(pde_t *to, pde_t *from, uintptr_t start, uintptr_t end, bool share)
{
    assert(start % PGSIZE == 0 && end % PGSIZE == 0);
    assert(USER_ACCESS(start, end));
    // copy content by page unit.
    do
    {
        // call get_pte to find process A's pte according to the addr start
        pte_t *ptep = get_pte(from, start, 0), *nptep;
        if (ptep == NULL)
        {
            start = ROUNDDOWN(start + PTSIZE, PTSIZE);
            continue;
        }
        // call get_pte to find process B's pte according to the addr start. If
        // pte is NULL, just alloc a PT
        if (*ptep & PTE_V)
        {
            if ((nptep = get_pte(to, start, 1)) == NULL)
            {
                return -E_NO_MEM;
            }
            uint32_t perm = (*ptep & PTE_USER);
            // get page from ptep
            struct Page *page = pte2page(*ptep);
            int ret = 0;

            // LAB5 CHALLENGE: Copy on Write 逻辑
            if (share)
            {
                // 如果页面是可写的，需要将其标记为只读，以便父子进程共享
                if (perm & PTE_W) {
                    perm &= ~PTE_W;
                    // 修改父进程的映射：去掉写权限
                    // page_insert 会自动处理引用计数(ref不变)并刷新TLB
                }
            }
        }
    } while (start < end);
    return ret;
}
```

```

        ret = page_insert(from, page, start, perm);
        if (ret != 0) return ret;
    }

    // 将该页面映射给子进程，权限与父进程一致（只读）
    // page_insert 会自动增加物理页的引用计数 (ref++)
    ret = page_insert(to, page, start, perm);
    assert(ret == 0);
}
else
{
    // --- 原有逻辑 (Deep Copy) ---
    // alloc a page for process B
    // 我们将分配移到了这里，只有在非共享模式下才分配新页
    struct Page *npage = alloc_page();
    assert(page != NULL);
    assert(npage != NULL);

    /* LAB5:EXERCISE2 2312220
     * replicate content of page to npage, build the map of phy addr of
     * nage with the linear addr start
     * Some Useful MACROs and DEFINES, you can use them in below
     * implementation.
     * MACROs or Functions:
     *   page2kva(struct Page *page): return the kernel virtual addr of
     *   memory which page managed (SEE pmm.h)
     *   page_insert: build the map of phy addr of an Page with the
     *   linear addr la
     *   memcpy: typical memory copy function
     */
    // 1. 获取源页面的内核虚拟地址
    void *kva_src = page2kva(page);
    // 2. 获取目标页面（新分配页）的内核虚拟地址
    void *kva_dst = page2kva(npage);
    // 3. 复制内存内容（复制一整页 4096 字节）
    memcpy(kva_dst, kva_src, PGSIZE);
    // 4. 建立物理地址与线性地址的映射
    ret = page_insert(to, npage, start, perm);
    assert(ret == 0);
}
}
start += PGSIZE;
} while (start != 0 && start < end);

```

```
    return 0;
}
```

2.2 修改 kern/mm/vmm.c (Page Fault 处理)

在 `do_pgfault` 中处理写保护异常。

```
int do_pgfault(struct mm_struct *mm, uint32_t error_code, uintptr_t addr) {
    int ret = -E_INVALID;
    struct vma_struct *vma = find_vma(mm, addr);

    pgfault_num++;

    if (vma == NULL || vma->vm_start > addr) {
        return -E_INVALID;
    }

    // 权限检查: 如果尝试写一个不可写的VMA, 直接报错
    if ((error_code & 2) && !(vma->vm_flags & VM_WRITE)) {
        return -E_INVALID;
    }

    uint32_t perm = PTE_U;
    if (vma->vm_flags & VM_WRITE) {
        perm |= (PTE_R | PTE_W);
    }
    if (vma->vm_flags & VM_READ) {
        perm |= PTE_R;
    }
    if (vma->vm_flags & VM_EXEC) {
        perm |= PTE_X;
    }

    addr = ROUNDDOWN(addr, PGSIZE);
    ret = -E_NO_MEM;
    pte_t *ptep = NULL;

    // 获取 PTE, 如果不存在(PT未分配)则分配
    if ((ptep = get_pte(mm->pgdir, addr, 1)) == NULL) {
        return ret;
    }

    // Case 1: 页表项全为0, 说明尚未建立映射 (Demand Paging)
```



```

if (*ptep == 0) {
    if (pgdir_alloc_page(mm->pgdir, addr, perm) == NULL) {
        return ret;
    }
}
// Case 2: 页表项存在, 可能是 COW 或者 Swap (本实验暂不考虑swap)
else {
    if (*ptep & PTE_V) {
        // LAB5 CHALLENGE: Copy on Write 处理
        // 判断条件: 这是写操作 (error_code & 2)
        //          && 且物理页目前是只读的 (!(*ptep & PTE_W))
        //          && 且 VMA 允许写入 (vma->vm_flags & VM_WRITE)
        if ((error_code & 2) && !(*ptep & PTE_W) && (vma->vm_flags & VM_WRITE)) {
            struct Page *page = pte2page(*ptep);

            // 情况 A: 页面被多个进程共享 (Reference Count > 1)
            // 需要执行“复制”: 分配新页, 拷贝内容, 重新映射
            if (page_ref(page) > 1) {
                struct Page *npage = alloc_page();
                if (npage == NULL) return ret;

                // 复制原页面内容到新页面
                memcpy(page2kva(npage), page2kva(page), PGSIZE);

                // =====
                // 【Dirty COW 模拟注入点】
                // =====
                if (TEST_DIRTY_COW_FLAG) {
                    cprintf("[DirtyCOW] ATTACK: Another thread clears the PTE now!\n");
                    *ptep = 0;
                    tlb_invalidate(mm->pgdir, addr);

                    // 模拟 madvise 的行为:
                    // 清空 PTE 的同时, 必须减少原物理页的引用计数。
                    // 这样, 如果攻击发生, 这个页面的引用计数就是正确的, 不会造成人工泄漏。
                    page_ref_dec(page);
                }
            }
            // =====

            // =====
            // [漏洞修复]
            // =====

```

```

// 在建立新映射前，再次检查 PTE 是否被修改。
// 如果 PTE 变成了 0 (被清空)，或者不再指向原来的 page，
// 说明发生了竞态条件，必须中止当前操作。
if ((*ptep & PTE_V) == 0 || pte2page(*ptep) != page) {
    cprintf("[DirtyCOW] Race detected! Retrying...\n");

    // 关键步骤：释放刚才分配的新页，否则这里本身也会导致泄漏
    free_page(npage);

    // 返回 0 表示“处理完毕”，CPU 会重新执行刚才那条写指令。
    // 届时会再次触发缺页异常，重新进入 do_pgfault 流程。
    return 0;
}
// =====

// 建立新映射：
// 1. page_insert 会把虚拟地址 addr 映射到 npage
// 2. npage 的 ref 会增加
// 3. 原 page 的 ref 会自动减少 (因为该地址原先映射的页被覆盖了)
// 4. 注意：这里赋予了 PTE_W 写权限
if (page_insert(mm->pgdir, npage, addr, perm) != 0) {
    return ret;
}
}
// 情况 B：页面只有当前进程在使用 (Reference Count == 1)
// 说明其他共享的进程已经退出，或者经过多次COW后只剩自己
// 只需要恢复写权限即可，无需拷贝
else {
    // page_insert 会检测到是同一个页，只更新权限并刷新 TLB
    if (page_insert(mm->pgdir, page, addr, perm) != 0) {
        return ret;
    }
}
} else {
    // 如果不是 COW 情况的权限错误，则返回错误
    return ret;
}
}
}
return 0;
}

```

2.3 修改 kern/mm/vmm.c (开启共享)

在 dup_mmap 中将 share 标志置为 1。

```
int dup_mmap(struct mm_struct *to, struct mm_struct *from) {
    // ...
    // 这里将 share 改为 1 (true)
    bool share = 1;
    if (copy_range(to->pgdir, from->pgdir, vma->vm_start, vma->vm_end, share) != 0) {
        return -E_NO_MEM;
    }
    // ...
}
```

三、测试验证

为了全面验证 Copy-on-Write 机制的正确性 (Correctness) 和有效性 (Efficiency)，我们设计了三个维度的测试用例：内存效率测试、数据隔离性测试以及多级 Fork 压力测试。

3.1 内存效率测试: cow_mem.c

测试目的: 验证 fork 时是否真的没有复制物理内存（仅复制页表）。

测试逻辑: 申请 10MB 大内存并填充，记录 fork 前后的空闲页数量差值。

```
#include <stdio.h>
#include <ulib.h>
#include <syscall.h>

#define SIZE (10 * 1024 * 1024) // 10MB 数据
volatile char big_mem[SIZE];    // 使用 BSS 段全局数组

int main(void) {
    cprintf("COW Memory Efficiency Test: Start\n");
    // 1. 强制分配物理页
    for (int i = 0; i < SIZE; i += 4096) big_mem[i] = 1;

    int free_before = sys_get_free_pages();
    int pid = fork();

    if (pid == 0) {
```

```

    volatile char temp = big_mem[0]; // 子进程只读
    exit(0);
} else {
    int free_after = sys_get_free_pages();
    int diff = free_before - free_after;
    cprintf("Pages consumed by fork: %d\n", diff);

    // 判定: 10MB 约需 2560 页。如果 diff 远小于此, 说明 COW 生效。
    if (diff < 1000) cprintf("COW Memory Efficiency Test: PASSED\n");
    else cprintf("COW Memory Efficiency Test: FAILED\n");
    waitpid(pid, NULL);
}
return 0;
}

```

测试结果分析:

```

COW Memory Efficiency Test: Start
Memory allocated and filled.
Free pages before fork: 28961
Store/AMO page fault      <-- 写栈触发缺页
Free pages after fork: 28948
Pages consumed by fork: 13
COW Memory Efficiency Test: PASSED
Explanation: Only page tables were copied, not physical memory.

```

• 结果解读:

- i. 父进程分配了 10MB 内存, 理论上如果进行深拷贝 (Deep Copy), 子进程创建后系统空闲页应减少约 2560 页。
- ii. 实际日志显示 Pages consumed by fork 仅为 **13** 页。
- iii. 这 13 页仅用于子进程的新页表 (Page Table) 和内核栈等管理结构。
- iv. 这有力地证明了物理内存没有被复制, 父子进程共享了物理页, COW 机制生效。

3.2 数据隔离性测试: cow_data.c

测试目的: 验证父子进程的数据在发生写操作后是否互不干扰 (正确性验证)。

测试逻辑: 父子进程共享变量, 子进程修改后, 验证父进程看到的数值是否保持原样。

```

#include <stdio.h>
#include <ulib.h>
int global_flag = 0;

int main(void) {
    cprintf("COW Data Isolation Test: Start\n");
    int pid = fork();
    if (pid == 0) {
        // 子进程写入，应触发 COW 复制
        global_flag = 1;
        cprintf("Child: global_flag = %d (Should be 1)\n", global_flag);
        exit(0);
    } else {
        waitpid(pid, NULL);
        // 父进程读取，应仍为旧值
        cprintf("Parent: global_flag = %d (Should be 0)\n", global_flag);
        if (global_flag == 0) cprintf("COW Data Isolation Test: PASSED\n");
    }
    return 0;
}

```

测试结果分析:

```

COW Data Isolation Test: Start
Store/AMO page fault      <-- 关键日志：子进程尝试写入 global_flag 触发缺页异常
Child: Writing to global_flag...
Store/AMO page fault
Store/AMO page fault
Child: global_flag = 1 (Should be 1)
Parent: global_flag = 0 (Should be 0)
COW Data Isolation Test: PASSED

```

• 结果解读:

- i. 日志中出现了 Store/AMO page fault，说明写入操作被内核捕获。
- ii. 内核发现该页面引用计数 > 1，执行了页面复制（Copy）并重新映射。
- iii. 最终子进程变量变为 1，而父进程变量保持为 0。
- iv. 证明了在共享物理页的情况下，写入操作正确触发了分离，保证了进程间的数据隔离。

3.3 多级 Fork 压力测试: cow_stress.c

测试目的: 验证引用计数 (Reference Counting) 逻辑在多级继承 (孙子进程) 下的正确性。

测试逻辑: 父 -> 子 -> 孙。验证孙子修改内存不影响子进程, 子进程修改不影响父进程。

```
#include <stdio.h>
#include <ulib.h>
int shared_arr[1000];

int main(void) {
    cprintf("COW Stress Test: Start\n");
    for(int i=0; i<1000; i++) shared_arr[i] = i;

    int pid1 = fork();
    if (pid1 == 0) { // 子进程
        int pid2 = fork();
        if (pid2 == 0) { // 孙子进程
            shared_arr[500] = 9999; // 孙子修改
            cprintf("Grandchild: arr[500] = %d\n", shared_arr[500]);
            exit(0);
        }
        waitpid(pid2, NULL);
        if (shared_arr[500] == 500) cprintf("Child: Data intact after Grandchild exit.\n");
        exit(0);
    }
    waitpid(pid1, NULL);
    cprintf("COW Stress Test: PASSED\n");
    return 0;
}
```

测试结果分析:

```
COW Stress Test: Start
Store/AMO page fault      <-- 多次触发缺页, 分别对应不同层级的写入
Store/AMO page fault
Store/AMO page fault
Store/AMO page fault
Store/AMO page fault
Grandchild: arr[500] = 9999
Child: Data intact after Grandchild exit.
Store/AMO page fault
COW Stress Test: PASSED
```

- **结果解读:**

- i. 孙子进程写入 9999 时触发 Page Fault，独立复制了页面。
- ii. Child: Data intact 证明孙子进程的修改没有污染子进程的数据（子进程仍读到 500）。
- iii. 最终 PASSED 证明父进程的数据也未受影响。
- iv. 这验证了内核中对 page->ref 的维护（从 3 -> 2 -> 1）以及共享链的处理是正确的，能够处理复杂的多级进程关系。

四、 Dirty COW 漏洞模拟与分析

4.1 漏洞原理与 ucore 现状分析

Dirty COW (CVE-2016-5195) 是 Linux 内核中一个利用 **Copy-on-Write (COW)** 机制竞态条件 (Race Condition) 的漏洞。

- **攻击原理:** 攻击者利用两个线程，一个尝试写入只读的 COW 映射页（触发缺页异常），另一个同时通过 `madvise(MADV_DONTNEED)` 丢弃该页面映射。如果不加锁，内核可能在“准备好新页面”和“更新页表”之间的时间窗口内，被另一个线程清空了页表项，导致后续逻辑出错（如错误地写入了原始文件页，或状态维护错误）。
- **ucore 现状:** 目前的 ucore Lab 5 难以自然复现该漏洞，因为：
 - i. 缺乏 `madvise` 系统调用。
 - ii. 用户进程为单线程模型，无法构造针对同一页表的并发操作。
 - iii. 内核缺页处理过程通常不可抢占。

4.2 模拟策略：白盒注入 (White-box Injection)

为了验证 COW 逻辑中原子性的重要性，我们采用“白盒注入”的方式模拟攻击。

我们在内核的 `do_pgfault` 函数中人为插入一段恶意逻辑：在内核**决定执行复制**之后，但**尚未建立新映射**之前，强行清空当前页表项（PTE）。

状态流转对比:

- **正常流程:** 检测到共享 -> 分配新页 -> 建立新映射 -> **检测到原位置有效** -> 减少旧页引用计数。
- **攻击流程:** 检测到共享 -> 分配新页 -> **[注入: PTE被清空]** -> 建立新映射 -> **检测到原位置为空** -> 跳过减少旧页引用计数。

预期后果: 旧的物理页引用计数发生**泄漏 (Reference Count Leak)**。即使所有进程退出，系统仍认为该页正在被使用，导致物理内存无法回收。

4.3 核心代码实现

1. 内核态：注入漏洞与控制开关

我们在 `kern/mm/vmm.c` 中添加全局开关 `TEST_DIRTY_COW_FLAG`，并在 `do_pgfault` 中注入破坏代码。同时添加系统调用 `SYS_set_cow_attack` (251) 用于用户态控制。

```
// kern/mm/vmm.c 中的核心修改逻辑
int do_pgfault(struct mm_struct *mm, uint32_t error_code, uintptr_t addr) {
    // ... (前置逻辑)
    if (page_ref(page) > 1) {
        struct Page *npage = alloc_page();
        memcpy(page2kva(npage), page2kva(page), PGSIZE);

        // =====
        // [模拟攻击注入]
        // 模拟：并发线程在此时刻调用了 madvise，导致 PTE 被清空
        if (TEST_DIRTY_COW_FLAG) {
            cprintf("[DirtyCOW] ATTACK: Another thread clears the PTE now!\n");
            *ptep = 0; // 强行清空 PTE，破坏状态
            tlb_invalidate(mm->pgdir, addr);
        }
        // =====

        // page_insert 尝试建立新映射
        // 由于 *ptep 在上面被恶意清空，page_insert 内部会认为该虚拟地址原本没有映射，
        // 因此它不会执行 page_ref_dec(old_page)。
        // 结果：old_page 的引用计数没有减少，产生泄漏。
        if (page_insert(mm->pgdir, npage, addr, perm) != 0) {
            return -E_NO_MEM;
        }
    }
    // ...
}
```

2. 用户态：三层进程测试模型

为了检测引用计数泄漏，我们设计了 `dirty_cow_test.c`，采用 **Grandparent -> Parent -> Child** 的三层结构：

1. **Grandparent (Main)**: 记录基准空闲页数量 (Baseline)，最后检查剩余空闲页 (Final)。
2. **Parent (Middle)**: 负责申请内存，持有物理页引用，等待子进程结束后**退出**（释放引用）。
3. **Child (Attacker)**: 开启攻击开关，写入内存触发 COW，导致内核逻辑错误。


```
// user/dirty_cow_test.c 核心逻辑
void run_victim_process() {
    volatile int data = 200; // 这里的写入会导致分配一个物理页(ref=1)
    int pid = fork();        // fork后, 该物理页变为共享(ref=2)
    if (pid == 0) {
        // --- 孙子进程 (Attacker) ---
        sys_set_cow_attack(1); // 开启内核中的破坏开关

        // 写入触发 do_pgfault -> 进入注入逻辑
        // 结果: 新页被映射, 但旧页的引用计数保持为 2 (未减少)
        data = 300;

        exit(0);
    }
    waitpid(pid, NULL);
    // Parent 退出, 释放它持有的引用 (ref 从 2 减为 1)
    // 正常情况下应减为 0 并回收, 但由于泄漏, 系统仍认为有 1 个引用
    exit(0);
}

int main() {
    int baseline = sys_get_free_pages();
    int pid = fork();
    if (pid == 0) run_victim_process();
    waitpid(pid, NULL); // 等待所有子/孙进程退出

    int final = sys_get_free_pages();
    // 如果 diff > 0, 说明物理页没有被系统回收
    if (baseline - final > 0)
        cprintf("RESULT: MEMORY LEAK DETECTED!\n");
}

```

4.4 攻防测试结果（攻击生效验证）

在未加入修复代码前, 执行 dirty_cow_test, 获得的实验日志如下:

```
Dirty COW Simulation: Start
Baseline free pages: 31512
...
[Kernel] Dirty COW Attack Simulation: ENABLED
[Child] Attacking...
Store/AMO page fault

```

```
[DirtyCOW] ATTACK: Another thread clears the PTE now! <-- 攻击注入成功
Store/AMO page fault
[Child] Done. Exiting.
[Middle] Child exited. Middle process exiting now.
Final free pages:    31511
Missing pages:      1                                <-- 物理页泄漏 1 页

RESULT: MEMORY LEAK DETECTED!
Explanation: The physical page was not freed after owner exited.
Dirty COW Simulation: SUCCESS (Bug Reproduced)
```

分析：测试开始时空闲页为 31512，结束时为 31511，丢失了 1 个物理页。这证实了我们在 `do_pgfault` 中模拟的竞态条件成功破坏了内核的引用计数维护逻辑。

4.5 解决方案与修复验证

4.5.1 修复策略：二次检查 (Double-Check)

Dirty COW 的核心问题在于“状态检查”和“状态更新”之间的不一致。虽然操作系统通常使用页表锁 (Lock) 来解决，但在 `ucore` 中，我们采用一种更轻量级的 **Double-Check** 机制来修复此漏洞。

修复逻辑：

在执行最终的 `page_insert` (建立映射) 之前，再次检查页表项 (PTE) 是否仍然指向原来的物理页。如果发现 PTE 被修改 (例如被清空)，则说明发生了竞态，必须中止当前操作。

4.5.2 修复代码实现

在 `do_pgfault` 的 `page_insert` 调用前插入检查逻辑：

```
// kern/mm/vmm.c 修复代码片段

// ... [模拟攻击代码块结束后] ...

// [漏洞修复代码] (The FIX)
// 在建立新映射前，再次检查 PTE 是否被修改
if ((*ptep & PTE_V) == 0 || pte2page(*ptep) != page) {
    cprintf("[DirtyCOW] Race detected! Retrying...\n");

    // 1. 关键：释放刚才分配的新页 (npage)
    // 因为我们决定不建立映射了，如果这里不释放，npage 就会造成新的内存泄漏
    free_page(npage);

    // 2. 返回 0 (成功)
```

```

// 这告诉 CPU 缺页处理已完成。CPU 会重新执行刚才那条 Store/Write 指令。
// 由于 PTE 已经被清空（或改变），重试指令会再次触发缺页异常，
// 内核将重新进入 do_pgfault，正确地按 Demand Paging 等逻辑处理。
return 0;
}

// 安全地建立新映射
if (page_insert(mm->pgdir, npage, addr, perm) != 0) {
    return -E_NO_MEM;
}

```

4.5.3 修复后验证

应用修复代码后，再次运行 `dirty_cow_test`，日志如下：

```

Dirty COW Simulation: Start
Baseline free pages: 31512
...
[DirtyCOW] ATTACK: Another thread clears the PTE now!    <-- 1. 攻击注入
[DirtyCOW] Race detected! Retrying...                    <-- 2. 修复代码生效，拦截异常
Store/AMO page fault                                     <-- 3. CPU 重试指令，正常处理
Final free pages:    31512
Missing pages:       0                                   <-- 4. 内存完好，无泄漏

RESULT: No leak detected. (Normal Behavior)
Dirty COW Simulation: FAILED (Attack Failed = Defense Success)

```

结论：日志显示 `Race detected! Retrying...`，证明修复逻辑成功捕获了 PTE 被清空的异常情况，阻止了错误的映射建立，并正确回收了临时资源。系统最终无内存泄漏，漏洞被成功修复。

Challenge2：用户程序的加载机制分析

5.1 加载时机

说明该用户程序是何时被预先加载到内存中的？

- 在 Lab 5 中，用户程序（如 `hello`, `exit`）并非在运行时从磁盘读取。
- 它们在 **编译内核时**，通过 Makefile 里的宏和汇编指令（`.incbin`），直接以二进制数据的形式链接到了 **内核镜像 (Kernel Image)** 的数据段中。
- 当系统启动时，内核被加载到物理内存，这些用户程序的二进制代码也随之驻留在物理内存的内核区域中。

- 当执行 `kernel_execve -> load_icode` 时，内核直接将这数据从内核空间 `memcpy` 到新进程的用户空间。

5.2 与常用操作系统 (Linux/Windows) 的区别

特性	ucore (Lab 5)	常用操作系统 (Linux/Windows)
存储介质	随内核镜像常驻物理内存。	存储在磁盘/SSD 的文件系统中。
加载方式	全量加载 (Deep Copy): 进程创建时，将代码段、数据段一次性从内核区复制到用户区。	按需分页 (Demand Paging): <code>exec</code> 时仅建立虚拟内存映射 (VMA)，不读取磁盘。
内存占用	即使程序不运行，其二进制镜像也占用物理内存。	只有运行且访问到的页面才占用物理内存；可被换出。
原因	简化实验设计: Lab 5 尚未完善文件系统和磁盘驱动。 为了专注于进程和内存管理逻辑，采用了这种“静态链接、内存直拷”的简易方式。	效率与资源管理: 现代软件体积巨大，无法一次性装入内存。 按需加载能显著减少启动时间和内存压力。

非常抱歉，我明白了。您是希望我输出 Markdown 源码，并且把 `![alt text](...)` 这些图片标记当作纯文本字符串保留在文档的对应位置中，不要让它们消失。

以下是为您调整排版后的 Markdown 源码：

六、 分支任务一：lab5 的 gdb 调试调用以及返回

6.1 调试步骤

- 在 lab5 代码目录中新开终端 1，并输入命令：

```
qemu-system-riscv64 -machine virt -nographic -bios default -device loader,file=bin/ucore.img,ad
```

此时会打印 OpenSBI 和内核初始化，然后卡住不再继续输出（正常现象）。

```
ubuntu@ubuntu-virtual-machine: ~/桌面/code
ubuntu@ubuntu-virtual-machine:~/桌面/code$ qemu-system-riscv64 -machine virt -no
graphic -bios default -device loader,file=bin/ucore.img,addr=0x80200000 -s -S

OpenSBI v0.4 (Jul  2 2019 11:53:53)

      ____
     /  __ \
    | |  | |__  _ __  _ __ | (___ | |_) | | | | | | |
    | |  | | ' \ / _ \ ' \ \___ \| _ < | |
    | |__| | |_) | __/ | | |___) | |_) | | |
     \____/| .__/ \____|_|_|_|_____/|_____/_____|
           | |
           |_|

Platform Name           : QEMU Virt Machine
Platform HART Features  : RV64ACDFIMSU
Platform Max HARTs      : 8
Current Hart            : 0
Firmware Base           : 0x80000000
Firmware Size           : 112 KB
Runtime SBI Version     : 0.1

PMP0: 0x0000000080000000-0x000000008001ffff (A)
PMP1: 0x0000000000000000-0xffffffffffffffff (A,R,W,X)
DTB Init
HartID: 0
DTB Address: 0x82200000
Physical Memory from DTB:
  Base: 0x0000000080000000
  Size: 0x0000000080000000 (128 MB)
  End:  0x0000000087ffffff
DTB init completed
(THU.CST) os is loading ...
```

2. 在目录中新开一个终端 1，输入命令：

```
make gdb
```

开启 gdb 调试看到输出：

```
The target architecture is assumed to be riscv:rv64
Remote debugging using localhost:1234
0x0000000000001000 in ?? ()
```

证明已经连接到端口 1234，以防万一我们给他手动确认一个端口，输入：

```
target remote localhost:1234
```

进行确认发现已被该进程占用。

```
ubuntu@ubuntu-virtual-machine:~/桌面/code$ make gdb
riscv64-unknown-elf-gdb \
  -ex 'file bin/kernel' \
  -ex 'set arch riscv:rv64' \
  -ex 'target remote localhost:1234'
GNU gdb (SiFive GDB 9.1.0-2020.08.2) 9.1
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "--host=x86_64-linux-gnu --target=riscv64-unknown-elf".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://github.com/sifive/freedom-tools/issues>.
Find the GDB manual and other documentation resources online at:
  <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word".
Reading symbols from bin/kernel...
The target architecture is assumed to be riscv:rv64
Remote debugging using localhost:1234
0x000000000000001000 in ?? ()
(gdb) target remote localhost:1234
A program is being debugged already. Kill it? (y or n) n
Program not killed.
```

3. 接下来我们加载用户符号并打系统调用断点，依次输入：

```
add-symbol-file obj/__user_forktest.out
add-symbol-file obj/__user_exit.out
add-symbol-file obj/__user_hello.out
```

加载三处符号（选项选 y 确定），然后在 syscall 中第 19 行（ecall 附近）加一个断点，输入：

```
break user/libs/syscall.c:19
```

```
(gdb) add-symbol-file obj/__user_forktest.out
add symbol table from file "obj/__user_forktest.out"
(y or n) y
Reading symbols from obj/__user_forktest.out...
(gdb) add-symbol-file obj/__user_exit.out
add symbol table from file "obj/__user_exit.out"
(y or n) y
Reading symbols from obj/ __user_exit.out...
(gdb) add-symbol-file obj/__user_hello.out
add symbol table from file "obj/__user_hello.out"
(y or n) y
Reading symbols from obj/ __user_hello.out...
(gdb) break user/libs/syscall.c:19
Breakpoint 1 at 0x800096: user/libs/syscall.c:19. (2 locations)
(gdb) continue
Continuing.
```

最后进行 `continue` 操作，发现终端 1 也会接着输出一部分，终端 2 继续输出到达断点前。


```
Special kernel symbols:
  entry  0xc020004a (virtual)
  etext  0xc02059ec (virtual)
  edata  0xc02c59b0 (virtual)
  end    0xc02c9e5c (virtual)
Kernel executable memory footprint: 808KB
memory management: default_pmm_manager
physcial memory map:
  memory: 0x08000000, [0x80000000, 0x87ffffff].
vapaofset is 18446744070488326144
check_alloc_page() succeeded!
check_pgdir() succeeded!
check_boot_pgdir() succeeded!
use SLOB allocator
kmalloc_init() succeeded!
check_vma_struct() succeeded!
check_vmm() succeeded.
++ setup timer interrupts
kernel_execve: pid = 2, name = "forktest".
Breakpoint
█
```

4. 那么现在我们距离 `ecall` 函数已经很近了，先进行几次 `si`，然后通过 `x/8i $pc` 看我们距离 `ecall` 还有多远（`8i` 就是目前的位置往后 8 个汇编语句），发现距离 `ecall` 还差 3 条，因此继续三次 `si`，我们就到达了 `ecall` 的前面。

```

Breakpoint 1, syscall (num=num@entry=2) at user/libs/syscall.c:19
19          asm volatile (
(gdb) si
0x00000000008000fa      19          asm volatile (
(gdb) si
0x00000000008000fc      19          asm volatile (
(gdb) si
0x00000000008000fe      19          asm volatile (
(gdb) x/8i $pc
=> 0x8000fe <syscall+38>:      ld      a3,56(sp)
    0x800100 <syscall+40>:      ld      a4,64(sp)
    0x800102 <syscall+42>:      ld      a5,72(sp)
    0x800104 <syscall+44>:      ecall
    0x800108 <syscall+48>:      sd      a0,28(sp)
    0x80010c <syscall+52>:      lw      a0,28(sp)
    0x80010e <syscall+54>:      addi    sp,sp,144
    0x800110 <syscall+56>:      ret
(gdb) si
0x0000000000800100      19          asm volatile (
(gdb) si
0x0000000000800102      19          asm volatile (
(gdb) si
0x0000000000800104      19          asm volatile (

```

5. 之后我们先打下内核 trap 入口断点以便观察 ecall 进入内核后的 CSR，输入：

```
break __alltraps
```

然后我们进行一步 si 执行 ecall，到达断点输出：

```
0xfffffffffc0200f14 in __alltraps () at kern/trap/trapentry.S:123
```

观察此时的 CSR，我们依次输入：

```

info registers scause
info registers sepc
info registers stval

```

```
info registers sstatus
```

得到输出：

```
scause      0x8  8 (U-mode 的 Environment call 的 scause 是 8)
sepc        0x800104 8388868 (ecall 的地址)
stval       0x0  0 (正常)
sstatus      0x8000000000046020  -9223372036854489056 (SPP = 0 代表从 U-mode 进入)
```

```
(gdb) break __alltraps
Cannot access memory at address 0xffffffffc0200f10
(gdb) break __trapret
Cannot access memory at address 0xffffffffc0200f80
(gdb) si
0xffffffffc0200f14 in __alltraps () at kern/trap/trapentry.S:123
123      SAVE ALL
(gdb) info registers scause
scause      0x8      8
(gdb) info registers sepc
sepc        0x800104 8388868
(gdb) info registers stval
stval       0x0      0
(gdb) info registers sstatus
sstatus      0x8000000000046020  -9223372036854489056
```

6. 还差一个 sret，我们再次打上断点 `break __trapret`，然后 `continue`，再进行 20-30 次 `si` 操作，得到输出：

```
__trapret () at kern/trap/trapentry.S:133
133      sret
```

即代表我们到达了 sret 之前。

```

(gdb) break __trapret
Breakpoint 2 at 0xfffffffffc0200f80: file kern/trap/trapentry.S, line 131.
(gdb) continue
Continuing.

Breakpoint 2, __trapret () at kern/trap/trapentry.S:131
131      RESTORE_ALL
(gdb) si
0xfffffffffc0200f82      131      RESTORE_ALL
(gdb) si
0xfffffffffc0200f84      131      RESTORE_ALL
(gdb) si
0xfffffffffc0200f88      131      RESTORE_ALL

```

```

(gdb) si
0xfffffffffc0200fd0      131      RESTORE_ALL
(gdb) si
0xfffffffffc0200fd2      131      RESTORE_ALL
(gdb) si
0xfffffffffc0200fd4      131      RESTORE_ALL
(gdb) si
__trapret () at kern/trap/trapentry.S:133
133      sret
(gdb) x/i $pc
=> 0xfffffffffc0200fd6 <__trapret+86>: sret

```

7. 我们直接一步 si 执行 sret 然后输入 p/x \$pc 查看 pc 值得到输出 \$2 = 0x800108 (pc=sepc+4 符合)，然后输入 x/i \$pc 查看当前用户态指令，得到输出：

```

=> 0x800108 <syscall+48>: sd a0,28(sp)

```

停在用户态 syscall.c:19 的 sd a0,28(sp)，也就是 ecall 后的第一条指令。最后我们输入指令 info registers sstatus 查看目前的 S-CSR 得到报错：

```

sstatus      Could not fetch register "sstatus"; remote failure reply 'E14'

```

但由于特权级已经由 S 态降到了 U 用户态，所以出现 E14 报错无法读取 S-CSR 完全正常，反向验证了特权级转移。

```

(gdb) si
0x0000000000800108 in syscall (num=3, num@entry=2) at user/libs/syscall.c:19
19      asm volatile (
(gdb) p/x $pc
$2 = 0x800108
(gdb) x/10i $pc
=> 0x800108 <syscall+48>:      sd      a0,28(sp)
0x80010c <syscall+52>:      lw      a0,28(sp)
0x80010e <syscall+54>:      addi    sp,sp,144
0x800110 <syscall+56>:      ret
0x800112 <sys_exit>: mv      a1,a0
0x800114 <sys_exit+2>:      li      a0,1
0x800116 <sys_exit+4>:      j       0x8000d8 <syscall>
0x800118 <sys_fork>: li      a0,2
0x80011a <sys_fork+2>:      j       0x8000d8 <syscall>
0x80011c <sys_wait>: mv      a2,a1
(gdb) info registers sstatus
sstatus      Could not fetch register "sstatus"; remote failure reply 'E14'
(gdb) Quit
(gdb)

```

至此，gdb 调试结束。

6.2 调试中遇到的问题以及与大模型的互动

1. 前期可能经常出现连接超时的问题，我们在第一步中的 `-s` 的功能就是开启 gdbserver 在 1234 端口监听，而后通过询问大模型，我们又补充了手动确认命令 `target remote localhost:1234` 来确保不会连接超时。另外第一步的 `-s` 有冻结 CPU 的作用，等到加载用户符号和打第一处断点后才 `continue` 继续运行确保稳定连接。

2. 起初为了观察 CSR 数据，大模型给的思路是打 pending 断点但都未命中，尝试了

`hb helper_raise_exception`、`hb riscv_raise_exception`、`hb raise_exception`、`hb do_interrupt`、`hb gen_helper_raise_exception` 等，于是推测实验中是通过内联优化路径进行的，没有调用这些函数，所以最后通过 `break __alltraps`，即给 `trapentry.S` 的所有 `trap` 入口打上断点以便获取 CSR 数据。

6.3 总结

- **ecall 和 sret 的关系：**

ecall 保存返回点到 `sepc`，切换到 S-mode 进入内核处理。

sret 从 sepc + 4 恢复执行，降级到 U-mode 返回用户态。

- **QEMU 处理流程：**

QEMU TCG 慢路径模拟：

ecall → 设置 scause/sepc，切换 S-mode，跳转 stvec

sret → pc = sepc + 4，恢复 U-mode

- **TCG 思考：**

特权指令走慢路径（helper 调用）软件模拟硬件行为，与普通指令快路径不同。与 lab2 页表异常慢路径类似。

七、分支任务二：lab2的QEMU 内存管理与 TLB 机制源码调试

实验概述

本实验通过 GDB 对 QEMU (v4.1.1, RISC-V 架构) 进行源码级动态调试。实验旨在深入理解操作系统底层的内存管理机制，具体涵盖：

1. **Slow Path:** 模拟硬件 MMU 的多级页表遍历流程。
2. **Fast Path:** 软件 TLB 的快速查找与 Miss 处理回溯。
3. **No Paging:** Machine Mode 下未开启分页时的地址映射行为对比。

任务一：使用 GDB 验证启动流程与 SV39 地址映射

为了从硬件模拟层面深入理解 RISC-V 的地址转换机制，我们采用了“三终端协同调试法”。这种方法允许我们一方面控制内核的执行（Terminal 3），另一方面直接监控 QEMU 模拟器的内部 MMU 行为（Terminal 2）。

1. 终端一：启动 QEMU 模拟器 (Server)

操作： 执行 `make debug`。

分析：

该指令启动了 OpenSBI (Supervisor Binary Interface) 和 ucore 内核，但添加了 `-s -S` 参数，使得 CPU 在复位向量处暂停（Freeze），等待调试器连接。

- **OpenSBI 输出：** 显示固件加载地址为 `0x80000000`，这是物理内存的起始位置。
- **PMP 配置：** 物理内存保护（Physical Memory Protection）已配置，允许全地址空间的访问。

```
xwy@DESKTOP-JUDTUK3:/mnt/d/OS/lab2/lab2/code$ make debug
```

```
OpenSBI v0.4 (Jul  2 2019 11:53:53)
...
Platform Name           : QEMU Virt Machine
Platform HART Features  : RV64ACDFIMSU
Current Hart            : 0
Firmware Base          : 0x80000000
...
PMP0: 0x0000000080000000-0x000000008001ffff (A)
PMP1: 0x0000000000000000-0xffffffffffffff (A,R,W,X)
```

2. 终端三：启动 GDB 客户端控制内核

操作：执行 `make gdb` 并输入 `c`。

分析：

- 该终端运行 `riscv64-unknown-elf-gdb`，连接到 QEMU 的 1234 端口。
- 输入 `c` (continue) 后，QEMU 解除冻结，开始执行内核代码。这会导致 CPU 尝试进行取指或访存，从而触发我们在终端二设置的 MMU 硬件断点。

```
xwy@DESKTOP-JUDTUK3:/mnt/d/OS/lab2/lab2/code$ make gdb
...
Remote debugging using localhost:1234
0x0000000000001000 in ?? ()
(gdb) c
Continuing.
```

3. 终端二：调试 QEMU 源码 (核心验证步骤)

操作：使用 `sudo gdb` 附加到 QEMU 进程，并在 `get_physical_address` 设置条件断点，拦截内核地址转换过程。

3.1 连接与断点设置

我们通过 `pgrep`: `pgrep -f qemu-system-riscv64` 找到 QEMU 进程 ID (27259)，附加后设置断点拦截虚拟地址大于 `0xc0000000` (内核空间) 的访问。

```
(gdb) attach 27259
(gdb) handle SIGPIPE nostop noprint (忽略干扰信号)
(gdb) b target/riscv/cpu_helper.c:get_physical_address if addr > 0xc0000000
```

```
Breakpoint 1 at ...  
(gdb) c
```

3.2 捕获地址转换 (SV39 逻辑验证)

断点触发，捕获到虚拟地址 `0xfffffffffc02000d8`。以下是单步调试 (n) 过程中的关键数据分析：

A. 虚拟地址 (VA) 确认

```
158      {  
(gdb) p/x addr  
$1 = 0xfffffffffc02000d8
```

- **分析：**这是内核入口点的虚拟地址。在 RISC-V 64位中，高位进行符号扩展，实际有效部分对应 `0xc02000d8`。

B. 获取根页表基址 (SATP)

```
184          base = get_field(env->satp, SATP_PPN) << PGSHIFT;  
(gdb) p/x base  
$2 = 0x1b880206000
```

- **分析：**从 `satp` 寄存器读取 PPN 并左移 12 位，得到一级页表 (Level 2 Page Table) 的物理地址。

C. 页表遍历与大页识别 (Huge Page)

```
237      for (i = 0; i < levels; i++, ptshift -= ptidxbits) {  
(gdb) p i  
$3 = 0
```

- **分析：**RISC-V SV39 有 3 级页表。这里 `i=0` 表示处于第一层循环 (Level 2, 最高级)。

D. 解析页表项 (PTE)

```
291          target_ulong updated_pte = pte | PTE_A |  
(gdb) p/x pte  
$6 = 0x200000cf
```

- **数值解析 `0x200000cf`：**
 - **低 8 位 (Flags):** `0xcf = 1100 1111` (二进制)。

- V=1 (Valid): 有效。
- R=1, W=1, X=1: 可读、可写、可执行。
- 高位 (PPN): 0x20000。
- **结论**: 由于 R/W/X 位同时置位, 这表示一个 **叶子节点 (Leaf Entry)**。因为是在 Level 2 (i=0) 发现的叶子节点, 说明这是一个 **1GB 的大页 (Gigabit Huge Page)**。

E. 物理地址 (PA) 计算结果

```

334             *physical = (ppn | (vpn & ((1L << ptshift) - 1))) << PGSHIFT;
(gdb) n
337             if ((pte & PTE_R) || ((pte & PTE_X) && mxr)) {
(gdb) p/x *physical
$8 = 0x80200000

```

- **计算验证**:
 - QEMU 成功计算出物理基地址为 0x80200000。
 - 结合偏移量 0xd8, 实际物理地址为 0x802000d8。
 - 这完美验证了 ucore 启动时的线性映射关系: Virtual Address - 0x40000000 = Physical Address (即 0xc0200000 -> 0x80200000)。

4. 实验结论

通过三个终端的配合, 我们成功“切入”了 QEMU 的执行流。日志证明, ucore 内核在启动阶段正确建立了 SV39 页表, 并利用 1GB 大页机制完成了内核空间的线性映射。QEMU 的 MMU 逻辑准确地将虚拟地址 0xc02000d8 转换为了物理地址 0x802000d8。

5.调试截图

```
● xwy@DESKTOP-JUDTUK3:/mnt/d/OS/lab2/lab2/code$ pgrep -f qemu-system-riscv64
27259
○ xwy@DESKTOP-JUDTUK3:/mnt/d/OS/lab2/lab2/code$ sudo gdb
[sudo] password for xwy:
GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-git
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
--Type <RET> for more, q to quit, c to continue without paging--
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word".
(gdb) attach 27259
Attaching to process 27259
[New LWP 27261]
[New LWP 27260]
warning: could not find '.gnu_debugaltlink' file for /lib/x86_64-linux-gnu/libglib-2.0.so.0
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
0x0000722cc231ba30 in __GI_ppoll (fds=0x64c2c3c09b90, nfds=7, timeout=<optimized out>,
    sigmask=0x0) at ../sysdeps/unix/sysv/linux/ppoll.c:42
warning: 42      ../sysdeps/unix/sysv/linux/ppoll.c: No such file or directory
(gdb) handle SIGPIPE nostop noprint
Signal      Stop      Print     Pass to program Description
SIGPIPE     No        No        Yes         Broken pipe
(gdb) b target/riscv/cpu_helper.c:get_physical_address if addr > 0xc0000000
Breakpoint 1 at 0x64c287505fc2: file /mnt/d/OS/lab2/lab2/code/qemu-4.1.1/target/riscv/cpu_helper.c, line 158.
(gdb) c
Continuing.
[Switching to Thread 0x722cc18e36c0 (LWP 27261)]
```

```

Thread 2 "qemu-system-ris" hit Breakpoint 1, get_physical_address (env=0x64c2c
3bc6050,
    physical=0x722cc18e21f0, prot=0x722cc18e21e4, addr=18446744072637907160, a
ccess_type=2,
    mmu_idx=1) at /mnt/d/OS/lab2/lab2/code/qemu-4.1.1/target/riscv/cpu_helper.
c:158
158     {
(gdb) p/x addr
$1 = 0xfffffffffc02000d8
(gdb) n
163         int mode = mmu_idx;
(gdb) n
165         if (mode == PRV_M && access_type != MMU_INST_FETCH) {
(gdb) n
171         if (mode == PRV_M || !riscv_feature(env, RISCV_FEATURE_MMU)) {
(gdb) n
177         *prot = 0;
(gdb) n
181         int mxr = get_field(env->mstatus, MSTATUS_MXR);
(gdb) n
183         if (env->priv_ver >= PRIV_VERSION_1_10_0) {
(gdb) n
184             base = get_field(env->satp, SATP_PPN) << PGSHIFT;
(gdb) p/x base
$2 = 0x1b880206000
(gdb) n
185             sum = get_field(env->mstatus, MSTATUS_SUM);
(gdb) n
186             vm = get_field(env->satp, SATP_MODE);
(gdb) n
187             switch (vm) {
(gdb) n
191                 levels = 3; ptidxbits = 9; ptesize = 8; break;
(gdb) n
223             CPUState *cs = env_cpu(env);
(gdb) n
224             int va_bits = PGSHIFT + levels * ptidxbits;
(gdb) n
225             target_ulong mask = (1L << (TARGET_LONG_BITS - (va_bits - 1))) - 1
;
(gdb) n

```

```

226         target_ulong masked_msbs = (addr >> (va_bits - 1)) & mask;
(gdb) n
227         if (masked_msbs != 0 && masked_msbs != mask) {
(gdb) n
231         int ptshift = (levels - 1) * ptidxbits;
(gdb) n
237         for (i = 0; i < levels; i++, ptshift -= ptidxbits) {
(gdb) n
238             target_ulong idx = (addr >> (PGSHIFT + ptshift)) &
(gdb) p i
$3 = 0
(gdb) n
239             ((1 << ptidxbits) - 1);
(gdb) n
238             target_ulong idx = (addr >> (PGSHIFT + ptshift)) &
(gdb) n
242             target_ulong pte_addr = base + idx * ptesize;
(gdb) p/x pte
$4 = 0x64c2879e2718
(gdb) n
244             if (riscv_feature(env, RISCV_FEATURE_PMP) &&
(gdb) n
245                 !pmp_hart_has_privs(env, pte_addr, sizeof(target_ulong),
(gdb) n
244             if (riscv_feature(env, RISCV_FEATURE_PMP) &&
(gdb) n
252             target_ulong pte = ldq_phys(cs->as, pte_addr);
(gdb) p/x pte
$5 = 0x64c2879e2718
(gdb) n
254             target_ulong ppn = pte >> PTE_PPN_SHIFT;
(gdb) n
256             if (!(pte & PTE_V)) {
(gdb) n
259             } else if (!(pte & (PTE_R | PTE_W | PTE_X))) {
(gdb) n
262             } else if ((pte & (PTE_R | PTE_W | PTE_X)) == PTE_W) {
(gdb) n
265             } else if ((pte & (PTE_R | PTE_W | PTE_X)) == (PTE_W | PTE_X))
{
(gdb) n

```

```

268         } else if ((pte & PTE_U) && (mode != PRV_U) &&
(gdb) n
273         } else if (!(pte & PTE_U) && (mode != PRV_S)) {
(gdb) n
276         } else if (ppn & ((1ULL << ptshift) - 1)) {
(gdb) n
279         } else if (access_type == MMU_DATA_LOAD && !(pte & PTE_R) ||
(gdb) n
283         } else if (access_type == MMU_DATA_STORE && !(pte & PTE_W)) {
(gdb) n
286         } else if (access_type == MMU_INST_FETCH && !(pte & PTE_X)) {
(gdb) n
292             (access_type == MMU_DATA_STORE ? PTE_D : 0);
(gdb) n
291         target_ulong updated_pte = pte | PTE_A |
(gdb) p/x pte
$6 = 0x200000cf
(gdb) n
295         if (updated_pte != pte) {
(gdb) n
333         target_ulong vpn = addr >> PGSHIFT;
(gdb) n
334         *physical = (ppn | (vpn & ((1L << ptshift) - 1))) << PGSHI
FT;
(gdb) p/x *physical
$7 = 0x0
(gdb) n
337         if ((pte & PTE_R) || ((pte & PTE_X) && mxr)) {
(gdb) p/x *physical
$8 = 0x80200000
(gdb) n
338         *prot |= PAGE_READ;
(gdb) n
340         if ((pte & PTE_X)) {
(gdb) n
341             *prot |= PAGE_EXEC;
(gdb) n
345         if ((pte & PTE_W) &&
(gdb) n
346             (access_type == MMU_DATA_STORE || (pte & PTE_D)))
{

```

```

347             *prot |= PAGE_WRITE;
(gdb) n
349             return TRANSLATE_SUCCESS;
(gdb) n
353         }
(gdb) n
riscv_cpu_tlb_fill (cs=0x64c2c3bbd640, address=18446744072637907160, size=0,
                  access_type=MMU_INST_FETCH, mmu_idx=1, probe=false, retaddr=0)
    at /mnt/d/OS/lab2/lab2/code/qemu-4.1.1/target/riscv/cpu_helper.c:453
453         if (mode == PRV_M && access_type != MMU_INST_FETCH) {
(gdb)

```

任务二：TLB 查找机制分析 (TLB Lookup Logic)

1. 调试目标

验证“先查 TLB，失败后再查页表”的逻辑。由于“查 TLB”是高频操作，无法直接断点，采用**“回溯法”**定位。

2. 具体调试步骤

1. **设置断点：**直接断在 TLB Miss 的处理函数上。

```

(gdb) b tlb_fill
(gdb) c

```

2. **查看调用栈：**断点触发后，查看是谁调用了 `tlb_fill`。

```

(gdb) bt

```

3. **切换栈帧：**进入调用者函数查看源码。

```

(gdb) frame 1
(gdb) list

```

3. 关键调试日志与分析

```

Thread 2 hit Breakpoint 2, tlb_fill (...)
(gdb) bt
#0  tlb_fill (...) at accel/tcg/cputlb.c:871

```

```
#1 store_helper (...) at accel/tcg/cputlb.c:1522 <-- [分析] 发现调用者是写辅助函数
```

```
(gdb) frame 1
```

```
(gdb) list
```

```
1519         if (!tlb_hit(tlb_addr, addr)) {           <-- [核心证据]
1520             if (!victim_tlb_hit(env, ...)) {
1521                 tlb_fill(env_cpu(env), ...);      <-- [分析] 断点位置
```

结论：

- **Fast Path:** 代码第 1519 行 `if (!tlb_hit)` 证明了 QEMU 首先尝试在软件 TLB 中查找（模拟硬件 Tag 比较）。
- **Slow Path:** 只有当 `tlb_hit` 返回 `false`（Miss）时，才会进入 `if` 内部调用 `tlb_fill`。这验证了标准硬件流程。

4.调试截图

```
● xwy@DESKTOP-JUDTUK3:/mnt/d/OS/lab2/lab2/code$ pgrep -f qemu-system-riscv64
1684
○ xwy@DESKTOP-JUDTUK3:/mnt/d/OS/lab2/lab2/code$ sudo gdb
[sudo] password for xwy:
GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-git
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
--Type <RET> for more, q to quit, c to continue without paging--
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
  <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
--Type <RET> for more, q to quit, c to continue without paging--
Type "apropos word" to search for commands related to "word".
(gdb) attach 1684
Attaching to process 1684
[New LWP 1686]
[New LWP 1685]
warning: could not find '.gnu_debugaltlink' file for /lib/x86_64-linux-gnu/libglib-2.0.so.0
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
0x00007bdc6591ba30 in __GI_ppoll (fds=0x5970835c3b90, nfd=7, timeout=<optimized out>,
  sigmask=0x0) at ../sysdeps/unix/sysv/linux/ppoll.c:42
warning: 42  ../sysdeps/unix/sysv/linux/ppoll.c: No such file or directory
(gdb) handle SIGPIPE nostop noprint
Signal      Stop      Print     Pass to program Description
SIGPIPE     No        No        Yes         Broken pipe
(gdb) b tlb_fill
Breakpoint 1 at 0x5970579fff75: file /mnt/d/OS/lab2/lab2/code/qemu-4.1.1/accel/tcg/cputlb.c, line 871.
(gdb) c
Continuing.
[Switching to Thread 0x7bdc64ee36c0 (LWP 1686)]
```



```

Thread 2 "qemu-system-ris" hit Breakpoint 1, tlb_fill (cpu=0x597083577640, addr=4096, size=0,
access_type=MMU_INST_FETCH, mmu_idx=3, retaddr=0)
at /mnt/d/OS/lab2/lab2/code/qemu-4.1.1/accel/tcg/cputlb.c:871
871      CPUTLBClass *cc = CPU_GET_CLASS(cpu);
(gdb) bt
#0  tlb_fill (cpu=0x597083577640, addr=4096, size=0, access_type=MMU_INST_FETCH, mmu_idx=3,
retaddr=0) at /mnt/d/OS/lab2/lab2/code/qemu-4.1.1/accel/tcg/cputlb.c:871
#1  0x0000597057a00689 in get_page_addr_code (env=0x597083580050, addr=4096)
at /mnt/d/OS/lab2/lab2/code/qemu-4.1.1/accel/tcg/cputlb.c:1032
#2  0x0000597057a250ef in tb_htable_lookup (cpu=0x597083577640, pc=4096, cs_base=0, flags=3,
cf_mask=4278714368) at /mnt/d/OS/lab2/lab2/code/qemu-4.1.1/accel/tcg/cpu-exec.c:339
#3  0x0000597057a2462d in tb_lookup_cpu_state (cpu=0x597083577640, pc=0x7bdc64ee2928,
--Type <RET> for more, q to quit, c to continue without paging--
cs_base=0x7bdc64ee2920, flags=0x7bdc64ee291c, cf_mask=4278714368)
at /mnt/d/OS/lab2/lab2/code/qemu-4.1.1/include/exec/tb-lookup.h:43
#4  0x0000597057a253c2 in tb_find (cpu=0x597083577640, last_tb=0x0, tb_exit=0, cf_mask=524288)
at /mnt/d/OS/lab2/lab2/code/qemu-4.1.1/accel/tcg/cpu-exec.c:406
#5  0x0000597057a25ce9 in cpu_exec (cpu=0x597083577640)
at /mnt/d/OS/lab2/lab2/code/qemu-4.1.1/accel/tcg/cpu-exec.c:731
#6  0x00005970579d7f3f in tcg_cpu_exec (cpu=0x597083577640)
--Type <RET> for more, q to quit, c to continue without paging--
at /mnt/d/OS/lab2/lab2/code/qemu-4.1.1/cpus.c:1435
#7  0x00005970579d87f8 in qemu_tcg_cpu_thread_fn (arg=0x597083577640)
at /mnt/d/OS/lab2/lab2/code/qemu-4.1.1/cpus.c:1743
#8  0x0000597057e5b8d7 in qemu_thread_start (args=0x59708358dcd0)
at util/qemu-thread-posix.c:502
#9  0x00007bdc6589caa4 in start_thread (arg=<optimized out>) at ./nptl/pthread_create.c:447
#10 0x00007bdc65929c6c in clone3 () at ../sysdeps/unix/sysv/linux/x86_64/clone3.S:78
(gdb) frame 1
#1  0x0000597057a00689 in get_page_addr_code (env=0x597083580050, addr=4096)
at /mnt/d/OS/lab2/lab2/code/qemu-4.1.1/accel/tcg/cputlb.c:1032
1032      tlb_fill(env_cpu(env), addr, 0, MMU_INST_FETCH, mmu_idx, 0);
(gdb) list
1027      CPUTLBClass *entry = tlb_entry(env, mmu_idx, addr);
1028      void *p;
1029
1030      if (unlikely(!tlb_hit(entry->addr_code, addr))) {
1031          if (!VICTIM_TLB_HIT(addr_code, addr)) {
1032              tlb_fill(env_cpu(env), addr, 0, MMU_INST_FETCH, mmu_idx, 0);
1033              index = tlb_index(env, mmu_idx, addr);
--Type <RET> for more, q to quit, c to continue without paging--

```

```

1034      entry = tlb_entry(env, mmu_idx, addr);
1035  }
1036  assert(tlb_hit(entry->addr_code, addr));
(gdb)

```

任务三：未开启分页时的行为对比 (Machine Mode)

1. 调试目标

验证在 Machine Mode (尚未开启分页) 下, QEMU 是否会跳过页表查找。

2. 具体调试步骤

1. **冻结启动**：修改 Makefile，在 qemu 启动命令中添加 -s 参数（Freeze CPU at startup）。

2. **启动与挂载**：

- 终端1：make qemu (此时窗口会黑屏等待)。
- 终端2：pgrep ... -> sudo gdb -> attach ...。

3. **断点设置**：

```
(gdb) b target/riscv/cpu_helper.c:get_physical_address
(gdb) c
```

4. **激活 CPU**：回到终端1，按下 Ctrl+A 后按 c 进入 monitor，输入 c 回车开始运行。

5. **验证跳过逻辑**：在终端2单步调试。

3. 关键调试日志与分析

```
Thread 2 hit Breakpoint 1, get_physical_address (...)
(gdb) n
165         if (mode == PRV_M && access_type != MMU_INST_FETCH) {
(gdb) p mode
$3 = 3          <-- [分析] 确认当前为 Machine Mode (PRV_M)

(gdb) n
171         if (mode == PRV_M || !riscv_feature(env, RISCV_FEATURE_MMU)) {
(gdb) n
172             *physical = addr;  <-- [核心证据] 直接恒等映射
(gdb) n
349             return TRANSLATE_SUCCESS;
```

结论：

日志显示，当 mode = 3 时，代码直接执行了 *physical = addr，**完全跳过了**任务一中复杂的 for 循环。这证明 QEMU 在 M 模式下采用恒等映射。

4.调试截图

```
● xwy@DESKTOP-JUDTUK3:/mnt/d/OS/lab2/lab2/code$ pgrep -f qemu-system-riscv64
4377
○ xwy@DESKTOP-JUDTUK3:/mnt/d/OS/lab2/lab2/code$ sudo gdb
[sudo] password for xwy:
GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-git
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

--Type <RET> for more, q to quit, c to continue without paging--
For help, type "help".
Type "apropos word" to search for commands related to "word".
(gdb) attach 4377
Attaching to process 4377
[New LWP 4379]
[New LWP 4378]
warning: could not find '.gnu_debugaltlink' file for /lib/x86_64-linux-gnu/libglib-2.0.so.0
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
0x0000778763d1ba30 in __GI_ppoll (fds=0x58bbe3af6b30, nfd=5, timeout=<optimized out>,
    sigmask=0x0) at ../sysdeps/unix/sysv/linux/ppoll.c:42
warning: 42  ../sysdeps/unix/sysv/linux/ppoll.c: No such file or directory
(gdb) handle SIGPIPE nostop noprint
Signal      Stop      Print     Pass to program Description
SIGPIPE     No        No        Yes        Broken pipe
(gdb) b target/riscv/cpu_helper.c:get_physical_address
Breakpoint 1 at 0x58bbd25c8fc2: file /mnt/d/OS/lab2/lab2/code/qemu-4.1.1/target/riscv/cpu_helper.c, line 158.
(gdb) c
Continuing.
[Switching to Thread 0x7787613fe6c0 (LWP 4379)]

Thread 2 "qemu-system-ris" hit Breakpoint 1, get_physical_address (env=0x58bbe3ab3010,
```

```
physical=0x7787613fd720, prot=0x7787613fd714, addr=4096, access_type=2, mmu_idx=3)
at /mnt/d/OS/lab2/lab2/code/qemu-4.1.1/target/riscv/cpu_helper.c:158
158      {
(gdb) n
163      int mode = mmu_idx;
(gdb) n
165      if (mode == PRV_M && access_type != MMU_INST_FETCH) {
(gdb) p mode
$1 = 3
(gdb) n
171      if (mode == PRV_M || !riscv_feature(env, RISCV_FEATURE_MMU)) {
(gdb) n
172          *physical = addr;
(gdb) n
173          *prot = PAGE_READ | PAGE_WRITE | PAGE_EXEC;
(gdb) n
174          return TRANSLATE_SUCCESS;
(gdb) n
353      }
(gdb) n
riscv_cpu_tlb_fill (cs=0x58bbe3aaa600, address=4096, size=0, access_type=MMU_INST_FETCH,
mmu_idx=3, probe=false, retaddr=0)
at /mnt/d/OS/lab2/lab2/code/qemu-4.1.1/target/riscv/cpu_helper.c:453
453      if (mode == PRV_M && access_type != MMU_INST_FETCH) {
(gdb)
```

实验总结：QEMU 模拟与真实硬件的区别

通过上述三个任务的调试，总结对比表如下：

特性	真实 CPU TLB (Hardware)	QEMU 模拟 TLB (Software)
存储内容	VPN -> PPN (虚拟页号到物理页号)	GVA -> HVA (Guest虚拟地址到Host虚拟地址)
未开启分页时	硬件旁路 (Bypass): 电路不经过 TLB， 直接发物理地址。	依然经过 TLB 逻辑： 调试证明 QEMU 即使跳过了页表查找，仍需调用 tlb_fill 将恒等映射填入软件 TLB， 因为必须转换成宿主机地址才能读写内存。
查找方式	电路并行比较	软件数组索引 + Victim Cache